Dynamic Speed Limit System for Accident Prevention

By

No.	Full Name	Registration Number	Degree Program
1	Wasike Hanani Elizabeth	2023/BSE/148/PS	Bachelor of Software Engineering
2	Okello David	2023/BSE/127/PS	Bachelor of Software Engineering
3	Ikayo Emmanuel	2023/BSE/050/PS	Bachelor of Software Engineering
4	Ainomujuni Yovan	2023/BSE/158/PS	Bachelor of Software Engineering
5	Muwonge Stuart	2023/BSE/097/PS	Bachelor of Software Engineering

A Research Report Submitted to the Faculty of Computing and Informatics for the Study Leading to a Project in Partial Fulfillment of the Requirements for the Award of the Degree of Bachelor of Software Engineering of Mbarara University of Science and Technology

Supervisor:

Dr ADONES RUKUNDO

Department of Software Engineering

May 2026

Declaration

We hereby declare that this report is written in partial fulfillment of the requirements of the Degree of Bachelor of Software Engineering of Mbarara University of Science and Technology. It is a result of our original research work and is presented without any reservations for examination.

We also hereby state that this is our own work and that it has not been submitted to any other institution for another degree or qualification. Throughout this work we have acknowledged all sources used in its compilation.

Students:

Wasike Hanani Elizabeth

_____

Signature

Okello David

_____

Signature

Ikayo Emmanuel

_____

Signature

Ainomujuni Yovan

_____

Signature

Muwonge Stuart

_____

Signature

Date

_____

Approval

The research work culminating into this report was conducted under guidance and supervision.

Signature: _____

Dr ADONES RUKUNDO
Supervisor
Department of Software Engineering

Date: _____

Acknowledgements

We wish to express our sincere gratitude to the Department of Software Engineering, Faculty of Computing and Informatics, Mbarara University of Science and Technology for the academic support and the opportunity to undertake this project.

We would like to acknowledge the guidance and encouragement of our supervisor, Dr ADONES RUKUNDO, and all the academic staff who provided constructive feedback during the development of this report.

We are grateful to our project team members for their collaboration and commitment throughout the design and implementation of the Dynamic Speed Limit System.

Finally, we thank our families and friends for their support and patience during the preparation of this report.

Dedication

We dedicate this work to all road users and their families, in the hope that advanced technologies will continue to enhance road safety and reduce the tragic toll of traffic accidents worldwide.

Abstract

Road accidents remain a serious public health and transport safety challenge, with over-speeding and failure to adapt to changing road conditions contributing significantly to crash risk and crash severity. Traditional static speed limits provide general legal guidance, but they do not automatically respond to dynamic factors such as adverse weather, reduced visibility, time of day, location type, school zones, construction zones, and other contextual risks. This research presents the design and implementation of a Dynamic Speed Limit System for Accident Prevention, a mobile-based application intended to support drivers with context-aware speed recommendations. The project followed an iterative prototyping methodology that included requirements analysis, system design, implementation, testing, and brief usability testing. The system was developed using the Flutter framework and implements GPS-based speed monitoring, a multi-factor speed calculation algorithm, configurable alert thresholds, trip handling, route recommendation logic, and safety information screens. The core algorithm considers weather condition, time of day, location type, visibility level, and base speed limit in order to compute a recommended speed. Provider state management, Geolocator, and permission management packages were used to support the application architecture. The results of the project include a functional mobile prototype with a real-time speed display, recommended speed display, warning messages, colour-coded feedback, settings, trip history, alerts, route recommendations, and educational pages. Preliminary testing showed that the prototype can demonstrate the intended concept by reducing recommended speeds under higher-risk conditions such as poor visibility, night driving, rain, school zones, and construction zones. Users were also able to identify the current speed, recommended speed, and over-speed warning messages from the interface. The findings show that mobile-based adaptive speed guidance can provide practical driver support in contexts where infrastructure-based variable speed limit systems may be difficult or expensive to deploy. The prototype demonstrates that GPS speed monitoring, contextual parameters, and simple alert mechanisms can be combined to improve driver awareness of unsafe speed choices. However, the current implementation still requires further improvement in live weather data integration, map-based road classification, persistent local storage, GPS filtering, and larger-scale field evaluation. In conclusion, the study contributes a practical and affordable software prototype for mobile-based adaptive speed guidance and accident prevention. The Dynamic Speed Limit System shows potential to support safer driving decisions by helping drivers compare their current speed with a recommended speed that reflects prevailing road and environmental conditions. Future work should focus on strengthening data accuracy, expanding real-world testing, and integrating automated weather and map-based context detection.

Table of Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Introduction

Road safety remains a critical and persistent challenge in global public health and transportation management. Despite decades of international efforts and technological advancement, road traffic accidents continue to cause millions of deaths and injuries annually, with profound economic and social consequences. This research project addresses a fundamental gap in contemporary road safety systems: the inability of existing speed management infrastructure to adapt dynamically to real-time environmental and contextual conditions. The Dynamic Speed Limit System for Accident Prevention represents an integrated technological solution that combines mobile computing, real-time data processing, and intelligent algorithms to enhance driver safety and reduce accident risks through context-aware speed recommendations.

This chapter provides a comprehensive introduction to the research undertaken, establishing the foundational context, articulating the research problem, and delineating the objectives and scope of the study. The chapter is structured to introduce the background and motivation for the research, present the specific problem being addressed, define clear research objectives and questions, establish the significance of the work, and define the boundaries of the study.

1.2 Background

Road accidents claim approximately 1.3 million lives annually according to the World Health Organization (WHO), with over-speeding responsible for approximately 30% of accidents worldwide [39]. In Uganda, this proportion escalates to 57% of road accidents attributed to over-speeding incidents [35], highlighting the region's acute vulnerability to speed-related crashes.

1.2.1 Uganda's Road Safety Context

Uganda experiences among the highest motor vehicle accident rates in Sub-Saharan Africa: 35,000+ incidents annually, 3,000+ fatalities, with 62% speed-related and 5-6% of GDP cost (USD 1.4B annually) [34]. Critical gaps: (1) no dynamic speed management, (2) 45-65% accident increase during rains despite unchanged limits, (3) 40% of fatalities at night (28% traffic), indicating poor visibility adaptation [23].

Indicator	**Value cited in this study**	**Interpretation for the project**
Annual road incidents	35,000+ incidents	Road crashes represent a recurring national safety problem requiring preventive interventions.
Annual fatalities	3,000+ fatalities	Road accidents have severe human consequences beyond vehicle damage and traffic disruption.
Speed-related accidents	Approximately 62%	Speed management is a relevant intervention area for accident prevention.
Night-time fatalities	Approximately 40%	Time of day and visibility should be considered when recommending safer speeds.

Table 1.1: Uganda road safety indicators motivating the Dynamic Speed Limit System

Source note: Values are drawn from the Uganda road safety sources cited in this chapter [23, 34]. They are used to justify the project focus on speed, visibility, and contextual driver feedback.

1.2.2 Existing Speed Management Systems: Limitations and Benchmarks

Contemporary speed management approaches employ four distinct technological strategies with varying performance and cost profiles. *Static Speed Limit Systems* yield 3-8% accident reduction but plateau as drivers adapt [43]. *Variable Message Signs (VMS)* achieve 12-18% violation reduction but cost USD 15,000-40,000 per installation with USD 2,000-5,000 annual maintenance, rendering them economically unfeasible for developing regions [9]. *Connected Vehicle Systems* demonstrate 22-28% violation reduction but require government coordination and exceed USD 50,000 per vehicle [29]. *Mobile-Based Driver Alert Systems* achieve 15-20% speeding reduction with USD 5-15 per user cost, enabling rapid deployment in resource-constrained regions [1].

1.2.3 Economic Justification: Cost-Benefit Analysis

Mobile-based systems cost USD 2-5 per user (vs. USD 50,000+ for connected vehicles), with 12-18% accident reduction yielding 150:1-250:1 benefit-to-cost ratio. At cost of USD 10,000 per accident prevented, 4,200-6,300 annual accidents prevented generates USD 42-63M in benefits [38].

Approach	Cost implication	Relevance to this project
Static roadside signs	Low initial cost, but limited contextual response	Useful for legal limits, but cannot adapt to weather, visibility, or time of day.
Variable message signs	High installation and maintenance cost	Effective but difficult to deploy widely in resource-constrained settings.
Connected vehicle infrastructure	Very high coordination and equipment cost	Promising but not immediately practical for ordinary drivers.
Mobile-based driver feedback	Low marginal deployment cost where smartphones are available	Fits the project goal of affordable, driver-level adaptive speed guidance.

Table 1.2: Cost and deployability comparison of speed management approaches

1.2.4 Transportation Policy and Regulatory Framework

Uganda's National Transport Policy Framework (2018-2030) prioritizes road safety and recognizes technology as enabler: current speed limits 40 km/h urban, 50-60 peri-urban, 80-100 highway. System provides demand-side driver behavior modification via real-time feedback, complies with data privacy regulations (non-personally-identifiable storage) [22, 27].

1.3 Problem Statement

Despite the availability of posted speed limits and road safety enforcement, drivers often continue to rely on fixed speed guidance even when weather, visibility, time of day, location type, and traffic context make those speeds unsafe. Static speed limits do not automatically adjust during rain, fog, night driving, construction-zone activity, school-zone movement, or other conditions that reduce safe stopping distance and driver reaction time. This creates a driver-level information gap in which the driver may know

the legal speed limit but may not receive practical guidance about the safer speed for the current conditions.

The problem is more serious in resource-constrained settings because infrastructure-based dynamic speed systems such as variable message signs require significant capital investment and maintenance. As a result, many high-risk road segments continue to depend mainly on fixed signs and manual enforcement. Existing approaches also provide limited personalized feedback to drivers about how their current speed compares with a context-appropriate recommended speed.

Therefore, the central problem addressed by this project is the lack of an affordable, mobile-based system that can recommend safer speeds and alert drivers when their current speed is unsafe for the prevailing environmental and road context. The Dynamic Speed Limit System is proposed to address this gap by combining GPS-based speed monitoring, contextual parameters, and immediate driver feedback.

1.4 Objectives

1.4.1 Main Objective

To design, develop, and evaluate a mobile-based Dynamic Speed Limit System that recommends safer driving speeds using contextual parameters such as weather condition, time of day, location type, visibility level, and current vehicle speed.

1.4.2 Specific Objectives

(i) To review existing tools, technologies, and methodologies used for vehicle speed monitoring, dynamic speed guidance, and driver alert systems.

(ii) To design a Dynamic Speed Limit System architecture that combines GPS speed monitoring, contextual parameters, and an algorithm for computing recommended safe speed.

(iii) To implement the designed system as a functional Flutter mobile application with speed monitoring, alerts, trip handling, settings, route recommendations, and safety information screens.

(iv) To test the prototype using controlled functional scenarios and brief user feedback in order to assess usability, clarity of alerts, and correctness of speed recommendations.

1.5 Research Questions

The research is structured around the following specific research questions:

(i) How can weather condition, time of day, location type, visibility level, and vehicle speed be combined to determine context-appropriate recommended speeds?

(ii) How can the recommended speed and alert information be presented in a mobile application so that drivers can understand it quickly?

(iii) To what extent does the implemented prototype meet its functional requirements for speed monitoring, alert generation, trip handling, route recommendation, and user interaction?

1.6 Project Significance

1.6.1 Road Safety Enhancement

The dynamic speed limit system addresses a fundamental and persistent problem in road safety by providing drivers with real-time, context-responsive guidance that accounts for the actual conditions they encounter, directly contributing to reduced accident risks and prevention of speed-related fatalities.

1.6.2 Data-Driven Decision Making

The system generates comprehensive datasets regarding road conditions, driver behavior, accident hotspots, and environmental factors, providing empirical foundation for evidence-based policy decisions in traffic safety and urban planning.

1.6.3 Technological Innovation and ITS Advancement

The project contributes to the broader field of Intelligent Transportation Systems development by demonstrating practical integration of mobile computing, GPS-based speed monitoring, contextual parameters, and algorithmic driver feedback for safety applications.

1.6.4 Accessibility and Affordability

Unlike expensive infrastructure-based systems requiring substantial capital investment in fixed installations, the proposed system leverages ubiquitous smartphone technology, making advanced speed management technology accessible and economically feasible

even in regions with limited transportation budgets, particularly benefiting developing nations like Uganda.

1.6.5 Scalability and Sustainability

Once developed, the system can rapidly scale to thousands of users with minimal marginal costs, and its sustainability improves over time as user networks grow and generate valuable behavioral and environmental datasets.

1.7 Scope of the Study

The scope of this study encompasses the following dimensions:

Geographic Scope: While the motivation includes observation of particularly high accident rates in Uganda, the system is designed to function in any geographic location where GPS positioning and weather data services are accessible.

Technical Scope: The research focuses on the design and implementation of an intelligent speed recommendation algorithm that processes real-time environmental data and delivers recommendations through a mobile application. The system does not involve redesign of road infrastructure or vehicle control systems, but rather operates at the driver information and decision-making level.

Functional Scope: The system encompasses vehicle speed monitoring through GPS, user-selected contextual parameters such as weather and visibility, algorithmic speed computation, generation of driver alerts and recommendations, trip handling, driving safety feedback, and prototype route recommendation logic. Live weather integration, full map-based road classification, and production-grade persistent storage remain outside the completed prototype and are treated as future work.

User Scope: The primary target user is any driver or vehicle operator with access to a smartphone or mobile device capable of running the Flutter application on Android or iOS operating systems.

Temporal Scope: The research encompasses system design, development, functional testing, and preliminary evaluation. The study does not extend to long-term longitudinal assessment of real-world accident reduction or comprehensive fleet-level deployment.

Non-Scope: The research does not include redesign of traffic infrastructure, modification of posted regulatory speed limits, integration with government enforcement systems, or comprehensive accident epidemiology study.

Chapter 2

Literature Review

2.1 Literature Review

This chapter presents a comprehensive review of academic and technical literature pertaining to the major concepts and technologies underlying the Dynamic Speed Limit System for Accident Prevention. The literature review synthesizes findings from peer-reviewed journals, conference proceedings, technical reports, and industry standards to establish the theoretical grounding and practical justification for the proposed system. Key conceptual domains addressed include: (1) road safety and accident prevention mechanisms; (2) Intelligent Transportation Systems architectures and technologies; (3) real-time data processing and sensor integration; (4) algorithmic approaches to multi-factor decision-making; (5) vehicle telematics and driver behavior modification; and (6) mobile-based intervention platforms for safety applications.

2.2 Road Safety and Speed-Accident Relationships

2.2.1 Epidemiology of Speed-Related Accidents

Speed is consistently identified across global road safety research as one of the most significant modifiable risk factors in accident causation. The World Health Organization establishes that speed impacts accident risk through two primary mechanisms: (1)

increased severity of crash outcomes at higher speeds due to physics-based kinetic energy factors, and (2) reduced driver reaction time and vehicle stopping distance relative to hazards [40]. The relationship between speed and crash severity follows a non-linear power function; increases in vehicle speed above safe limits produce exponentially greater increases in crash severity rather than linear relationships [11].

Taylor et al. (2017) conducted a systematic meta-analysis of 97 studies examining speed-crash relationships across diverse contexts and found robust evidence that for every 1% increase in mean traffic speed, there is approximately a 4% increase in fatal crash risk and 2% increase in serious injury crash risk [33]. This relationship demonstrates particular strength in urban environments and on roads with vulnerable road users (pedestrians, cyclists).

Research by Litman (2014) documents that during adverse weather conditions, the speed-safety relationship becomes dramatically more pronounced. Rain reduces tire-road friction by 20-40% depending on surface type and rainfall intensity, effectively reducing safe operating speeds by 10-30%. Similarly, fog reduces visibility, with visibility reductions of 50% requiring speed reductions of approximately 25-40% to maintain constant safety margins [21].

2.2.2 Dynamic Speed Adaptation and Safety

Contemporary research in road safety increasingly recognizes that static speed limits, designed for optimal conditions on a specific road section, are inherently suboptimal during non-ideal conditions. Hydén and Varhelyi (2000) demonstrated through field studies that drivers, when provided with real-time information about actual road and environmental conditions, voluntarily reduce speeds to match conditions, without requirement for enforcement intervention [17].

The concept of "safe system" speed management, articulated by Gitelman et al. (2010), posits that speed recommendations should dynamically reflect the actual conditions encountered, the capabilities of the roadway and vehicles present, and the vulnerability of exposed populations. This framework directly supports development of dynamic speed systems that adjust recommendations based on real-time environmental assessment [14].

Risk factor	**Effect on driving safety**	**Speed recommendation implication**
Higher travel speed	Reduces reaction time and increases crash severity	Recommended speed should reduce when contextual risk increases.
Rain or wet road surface	Reduces tyre-road friction and increases stopping distance	Weather condition should lower the recommended speed.
Fog or poor visibility	Reduces the distance within which hazards can be seen	Visibility level should strongly influence speed guidance.
Night driving	Reduces visibility and can increase driver fatigue risk	Time-of-day should be included in the algorithm.

Table 2.1: Road safety factors considered in the speed recommendation model

2.3 Intelligent Transportation Systems and Real-Time Data Integration

Intelligent Transportation Systems integrate communications-based technologies with land transport infrastructure to enhance safety, mobility, and user service (ISO 2017). Variable Speed Limit (VSL) systems—a specific ITS category—dynamically adjust speed recommendations based on real-time conditions. Foundational research by Papageorgiou et al. (1997) demonstrated that VSL approaches significantly improve traffic flow and reduce accidents compared to static limits. Field trials show quantifiable benefits: German highways achieved 8-15% accident reduction and 5-12% traffic flow improvement (SPEED project 2008); Nordic implementations documented 10-20% se-

vere accident reduction. Del Serrone et al. (2025) provide algorithmic frameworks incorporating sight distance, stopping distance, and dynamic environmental adjustment—methodologies directly applicable to multi-factor speed computation [9, 12, 18, 24, 26].

2.4 Real-Time Environmental Data Collection and Processing

GPS technology achieves 5-10 meter horizontal accuracy and ±0.18 km/h velocity accuracy, sufficient for detecting speed violations and determining road context. Integration with inertial measurement units and map-matching algorithms improves accuracy and enables context interpretation (Alt et al. 2003). Modern weather services (OpenWeatherMap, National Weather Service API) provide real-time meteorological data (temperature, precipitation, visibility, wind) at 15-60 minute intervals. Research demonstrates that precipitation, visibility reduction, road surface conditions, and temperature significantly impact accident occurrence and severity (Andrey et al. 2003). Sensor fusion methodologies combine GPS positioning, vehicle dynamics, weather information, temporal factors, and location context into comprehensive datasets for decision-making, accounting for measurement uncertainty and data quality variations (Brynielsson et al. 2013) [2, 4, 7, 25, 36].

2.5 Algorithmic Approaches and Data Integration

Multi-factor speed computation requires systematic integration of influential factors using weighted averaging methodologies. Summala (1996) presents driver behavior models

where speed is determined by road conditions, weather, visibility, and vehicle capabilities, supporting algorithmic approaches that compute appropriate speeds as a function of these factors. Safe operating speeds are fundamentally limited by: (1) sight distance relative to obstacles, (2) stopping distance requirements, and (3) tire-road friction constraints. Jonasson and Brüde (2007) provide mathematical frameworks for computing speed limits ensuring stopping distance adequacy under varying visibility and weather conditions [20, 31].

Real-time driver feedback improves speed compliance: immediate feedback produces 10-20% reduction in speeding behavior, persisting 4-8 weeks after removal (Dozza et al. 2011). Visual feedback combined with normative guidance ("exceeding recommended speed") proves most effective; mobile applications with historical tracking and social comparison features achieve 15-25% sustained speeding reduction (Sundar et al. 2015). Context-specific speed recommendations improve through automatic road classification (highway/urban/residential) using GPS data and crowdsourced databases; drivers typically reduce speeds 10-20% in perceived high-risk zones (Yanagisawa and Swait 2010). Temporal factors—traffic flow, visibility, alertness—vary substantially across day/night and seasons; night-time accidents show higher severity patterns (Stathopoulos and Karlaftis 2003), supporting temporal information incorporation into speed recommendations [10, 30, 32, 41].

2.6 Mobile Technology Platforms for Safety Applications

Flutter framework enables rapid cross-platform mobile development for Android and iOS using a single codebase, achieving performance comparable to native applications while reducing development time and cost. Cloud-based platforms provide infrastructure for real-time data processing, storage, and API integration, with frameworks addressing

latency, reliability, and scalability considerations (Jayakrishnan et al. 2001) [15, 19].

2.7 Critical Review of Related Systems

Table 2.2 summarizes key literature and systems reviewed for this project. The table focuses not only on what each study contributes, but also on the limitations that shaped the proposed Dynamic Speed Limit System.

Author/System	**Method**	**Strength**	**Limitation**	**Gap Identified**	**How This System Responds**
Elvik (2005) speed-risk model [11]	Relates speed changes to crash and injury risk.	Provides strong theoretical evidence that speed reduction improves safety.	Does not provide a mobile implementation for driver feedback.	Need to translate speed-risk theory into a usable driver tool.	Uses contextual risk factors to recommend lower speeds when risk increases.
Papageorgiou motorway control [26]	Adaptive control for traffic flow and speed management.	Demonstrates value of dynamic traffic control.	Focuses mainly on motorway/infrastructure control systems.	Developing contexts need lower-cost approaches.	Implements driver-level guidance through a mobile prototype.
Del Serrone et al. VSL algorithm [9]	Uses road geometry, sight distance, stopping distance, and weather.	Strong engineering basis for dynamic speed limits.	Requires detailed road geometry and infrastructure data.	Lightweight systems need simpler deployable models.	Uses a simplified multi-factor model suitable for smartphone implementation.
Andrey et al. weather-risk studies [4]	Reviews weather effects on driving risk.	Shows that rain, visibility, and surface conditions affect safety.	Does not specify a complete mobile warning workflow.	Weather risk must reach the driver at the point of decision.	Includes weather and visibility as speed recommendation parameters.
Dozza et al. driver feedback [10]	Studies in-vehicle feedback and driver behaviour.	Shows that timely feedback can influence safer driving.	Feedback systems can distract if interface is poorly designed.	Need simple, readable alerts for mobile use.	Uses large speed display, colour feedback, and short warning messages.
Mobile safety applications [32]	Uses mobile interfaces for driver behaviour support.	Affordable and scalable compared to roadside systems.	May lack accurate contextual road interpretation.	Mobile apps need better context-aware speed logic.	Combines GPS speed tracking with contextual parameter-based recommendations.

Table 2.2: Critical review of related literature and systems

2.8 Gaps in Current Literature and Contribution of Present Research

While substantial literature exists on individual components addressed in the present research—variable speed limits, GPS-based monitoring, real-time weather integration, driver feedback systems—there is limited research specifically addressing integrated systems that combine these elements in accessible, affordable, mobile-platform implementations suitable for developing-region deployment.

Existing VSL research in developed nations (Europe, North America, Australia) typically assumes infrastructure-based implementations (variable message signs, connected vehicles) requiring substantial capital investment and government coordination. Mobile-based approaches, while studied in developing contexts, have not been comprehensively evaluated in combination with dynamic algorithmic speed computation incorporating all relevant contextual factors.

The present research addresses this gap by developing and implementing an integrated system specifically designed for mobile platform deployment, developing algorithmic frameworks for multi-factor speed computation, incorporating empirically validated driver feedback mechanisms, and evaluating the system within a developing-region context (Uganda) where road safety interventions carry particular significance.

Chapter 3

Methodology

This chapter describes the research design, development methodology, system architecture, tools and technologies, development timeline, and evaluation approach employed in the Dynamic Speed Limit System project. The chapter establishes the systematic framework through which research objectives are addressed, clarifies the development process adopted for the system implementation, and delineates the methodological rigor applied to testing and validation activities.

3.1 Research Design and Approach

The research design combines three complementary methodological frameworks to address the multifaceted requirements of developing a working prototype while rigorously evaluating its effectiveness:

3.1.1 Design Science Research

The primary methodological framework is Design Science Research (DSR), which is particularly well-suited for research that involves designing and building innovative artifacts (software systems, algorithms, prototypes) to solve real-world problems. DSR follows a structured iterative process of design, implementation, and evaluation, with each cycle incorporating feedback to refine the artifact [37]. This approach aligns with the project

objective of designing a novel system (dynamic speed algorithm, mobile application, backend infrastructure) that addresses documented safety gaps. The DSR approach includes six iterative design cycles, each incorporating requirements refinement, design modification, implementation, preliminary testing, and stakeholder feedback.

3.1.2 Case Study Evaluation Framework

A case study approach is adopted for system evaluation within the Uganda context. This framework allows for in-depth investigation of the system's real-world performance within a specific, well-documented environment (Uganda's road safety context, documented in Chapter 1). Case study methodology is appropriate for evaluating technological innovations in specific geographic and social contexts, allowing assessment of both technical performance and contextual factors affecting implementation [42].

3.1.3 Iterative Prototyping

The development process incorporates iterative prototyping, wherein each development cycle produces an executable prototype representing progressively more complete system functionality, subject to testing and refinement. This approach supports rapid feedback collection and allows design decisions to be validated against actual functional requirements rather than theoretical assumptions alone [16].

3.2 Development Methodology: Hybrid Agile with Kanban

3.2.1 Methodology Overview

The project employs a Hybrid Agile Development approach combining Agile principles with formal Kanban workflow management. This hybrid approach balances Agile's emphasis on iterative delivery, stakeholder engagement, and responding to change with Kanban's focus on continuous flow, visualized work queues, and work-in-progress (WIP) limits [28]. This combination is particularly well-suited for small development teams (5 members) with limited centralized project management infrastructure.

Iteration stage	**Project activity**
Problem and requirement analysis	Identified speed-related safety problem, target users, functional requirements, and non-functional requirements.
Design	Defined the mobile application screens, service structure, data models, and speed recommendation logic.
Implementation	Built the Flutter prototype with GPS speed tracking, parameter-based speed recommendation, alerts, trip models, settings, and route recommendation logic.
Testing and review	Tested controlled scenarios, alert thresholds, GPS permission behaviour, screen navigation, and preliminary usability feedback.
Refinement	Cleaned the interface, adjusted report claims to match implementation, and documented limitations for future work.

Table 3.1: Iterative development process followed in the project

3.2.2 Development Phases

The development lifecycle is structured into three distinct phases, each with specific objectives, activities, and deliverables:

3.2.2.1 Phase 1: Prototyping and Feasibility Analysis (Weeks 1-4)

Objectives: Establish project requirements, validate technical feasibility of core components, develop initial proof-of-concept prototype, gather preliminary design requirements.

Key Activities:

- Requirements gathering and technology assessment (Flutter framework, GPS speed tracking, contextual parameters, and future API options)
- Proof-of-concept GPS module development and feasibility testing (GPS accuracy, real-time processing, battery efficiency)
- Preliminary system architecture design with supervisor feedback

Deliverables:

- Requirements specification document
- Proof-of-concept prototype demonstrating GPS speed tracking accuracy
- System architecture diagram and feasibility assessment report

Timeline: Weeks 1-4 (Planning Phase: Weeks 1-2; Design and POC Phase: Weeks 3-4)

3.2.2.2 Phase 2: Agile Development and Implementation (Weeks 5-11)

Objectives: Implement complete system functionality across all layers, integrate external data sources, develop multi-factor speed algorithm, implement user interface and alert system, conduct iterative testing and refinement.

Development Approach - Agile Kanban Cycles: This phase employs iterative development cycles utilizing the Kanban methodology. Work items are organized into a Kanban board with columns representing workflow stages: Backlog → Selected → In Development → Testing → Code Review → Approved → Deployed. Each cycle targets 1-2 week duration with specific feature completion and testing objectives.

Development Cycles Structure:

Cycle 1 (Weeks 5-6): Data Acquisition and Processing

- GPS/location acquisition, weather and visibility parameter handling, and location-type selection
- Data preprocessing with unit and integration testing

Cycle 2 (Weeks 7-8): Speed Calculation Algorithm and Processing

- Implement multi-factor speed calculation algorithm with four weighting factors (weather, time, location type, visibility)
- Develop algorithm parameter calibration based on reviewed literature values
- Implement alert threshold setting and severity classification
- Test algorithm output using controlled scenarios for clear, rainy, night-time, urban, highway, school-zone, and construction-zone conditions

Cycle 3 (Weeks 9-10): User Interface, Alerts, and Trip Features

- Develop application user interface with real-time speedometer display
- Implement color-coded visual feedback system (green/yellow/red risk indicators)
- Develop notification/alert generation system with multiple notification channels
- Implement trip history tracking and data persistence
- Develop settings interface allowing user configuration
- Usability testing with target user representations

Cycle 4 (Week 11): Integration, Review, and Refinement

- Integrate GPS speed service, speed calculator, alert service, trip service, and route service
- Conduct end-to-end application testing across the main screens
- Stakeholder review and feedback incorporation
- Final refinement based on review comments

Kanban WorkFlow Management: The Kanban methodology provides visual workflow management. Each work item progresses through columns: Backlog → Selected → In Development (WIP limit: 3 items) → Testing → Code Review (WIP limit: 2) → Approved → Deployed. Daily standups (20 minutes) review column status, identify blocked items, and reallocate resources. Team members move items through columns as work progresses, maintaining visualization of project status [3].

Continuous Integration and Testing and Optimization

- Full-system integration testing across all layers with performance optimization
- Bug correction, automated testing implementation, stakeholder review, and final refinement
- User interface prototype and usability assessment
- System architecture diagram (final)
- Code documentation and developer guide

3.2.2.3 Phase 3: System Testing, Evaluation, and Finalization (Week 12)

Objectives: Conduct comprehensive system testing in simulated and real-world conditions, evaluate system performance, finalize documentation, prepare deliverables and presentation.

Key Activities:

- Comprehensive end-to-end system testing in controlled and simulated environments
- Performance metrics collection (GPS accuracy, processing latency, battery consumption, algorithm accuracy)
- User acceptance testing and final documentation compilation
- Stakeholder review, approval, and project presentation

Testing Methodology (detailed in Section 3.5)

Deliverables (Phase 3):

- System testing and evaluation report with performance metrics and benchmarks

- Final technical documentation, user manuals, and deployment guide
- Project presentation materials

Timeline: Week 12 (concentrated finalization week following development completion)

3.3 System Architecture and Design

The Dynamic Speed Limit System architecture follows a layered design pattern separating concerns into distinct functional layers, each responsible for specific aspects of system functionality. This architecture supports modular development, testability, and future scalability.

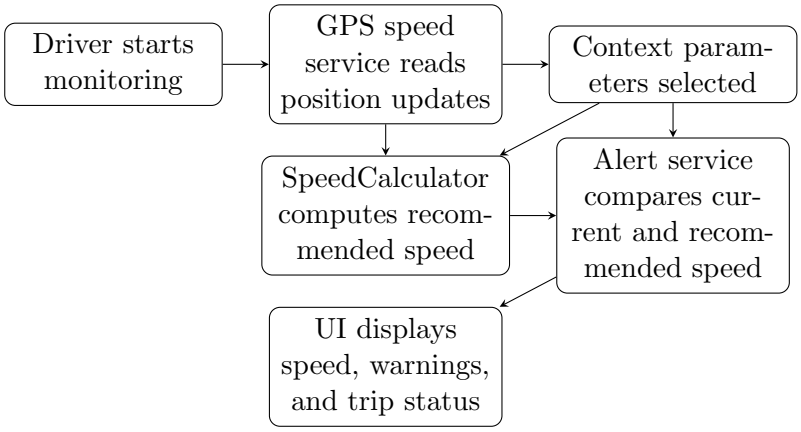


Fig. 3.1: Dynamic Speed Limit System process flow

3.3.1 Data Acquisition Layer

The Data Acquisition Layer combines implemented device data with contextual parameters selected in the prototype. Some data sources are implemented directly, while others are documented as future integrations.

- **GPS/Location Module**: Utilizes device GPS hardware via Geolocator package to obtain vehicle location (latitude, longitude) at 5-meter distance filter intervals, typically generating 2-5 location updates per second while driving. GPS accuracy specification is ±5-10 meters for position, ±0.18 km/h for velocity [36].

- **Speed Module**: Derives vehicle speed from GPS position changes and elapsed time. The current implementation does not use accelerometer fallback.

- **Weather Parameter Module**: Represents weather condition as a user-selected parameter in the current prototype. OpenWeatherMap integration is documented as future work, not as a completed feature.

- **Visibility Module**: Estimates visibility based on: (a) time of day (determines expected ambient lighting), (b) weather condition (rain/fog reduces visibility), (c) phone light sensor data (optional hardware sensor providing ambient light measurement).

- **Location Type Module**: Represents road context such as highway, urban, residential, school zone, or construction zone. In the current prototype this is selected within the application; map-based classification is future work.

- **Time Module**: Obtains current time from device clock, used to determine time-of-day factor (morning 6-9am, afternoon 10-16, evening 17-19, night 20-5) for algorithm weighting.

3.3.2 Data Processing and Storage Layer

This layer handles data quality management and persistent storage:

- **Data Preprocessing**: The design requires validation of incoming data for completeness and accuracy. The current prototype handles permission errors and

basic GPS position updates; advanced GPS outlier filtering and cached weather fallback remain future improvements [6].

- **Data Cleaning**: Advanced interpolation, offline queuing, and network recovery are not completed in the current source code and are therefore treated as future improvements.

- **Local Trip Handling**: Trip objects and alert records are represented in the application service and data model classes. The current source includes JSON serialization support but does not yet implement SQLite persistence.

- **Future Cloud/Server Storage**: Cloud synchronization is not part of the completed prototype. If added later, it should store only anonymized trip statistics and require explicit user consent.

3.3.3 Core Processing Layer

The Core Processing Layer implements the primary algorithm logic:

- **Safe Speed Calculation Algorithm**: Receives validated data inputs (weather, time, location type, visibility, current speed) and computes recommended safe speed through multi-factor weighted averaging (detailed in subsection 3.3.3.1).

- **Alert Generation Logic**: Compares current speed against recommended speed; if current speed exceeds recommended speed by threshold amount (typically 5-10 km/h), generates alert event with severity level.

- **Learning Model Integration**: Architecture includes provision for machine learning model integration. Current version uses static weighting parameters; future versions could incorporate learned weights from actual accident or safety databases.

- **Historical Trend Analysis**: Aggregates historical data to identify patterns (e.g., "highway segments where accidents cluster during rain"), providing input for future algorithm refinement.

3.3.3.1 Multi-Factor Speed Calculation Algorithm

The core speed recommendation algorithm integrates four primary factors through weighted averaging:

Formula:

$$\text{RecommendedSpeed} = \text{BaseSpeed} \times \frac{W_{\text{weather}} \times F_{\text{weather}} + W_{\text{time}} \times F_{\text{time}} + W_{\text{location}} \times F_{\text{location}} + W_{\text{visib}}}{W_{\text{total}}}$$

Where:

- BaseSpeed = Legal speed limit for road segment (from road database or user input)
- $W_{\text{weather}}, W_{\text{time}}, W_{\text{location}}, W_{\text{visibility}}$ = Weighting factors (currently: 40%, 20%, 20%, 20% respectively) [9]
- $F_{\text{weather}}, F_{\text{time}}, F_{\text{location}}, F_{\text{visibility}}$ = Factor multipliers (0.3 to 1.0 depending on conditions)

Weather Factor Multipliers:

- Clear conditions: $F_{\text{weather}} = 1.0$ (base speed)
- Partly cloudy: $F_{\text{weather}} = 0.95$
- Rain: $F_{\text{weather}} = 0.75$ (25% speed reduction)

- Heavy rain: $F_{\text{weather}} = 0.65$ (35% reduction)
- Snow/sleet: $F_{\text{weather}} = 0.50$ (50% reduction)
- Fog: $F_{\text{weather}} = 0.55$ (45% reduction)
- Thunderstorm: $F_{\text{weather}} = 0.40$ (60% reduction)

Time-of-Day Factor Multipliers:

- Morning (6am-9am, peak commute): $F_{\text{time}} = 0.90$ (10% reduction for increased traffic)
- Afternoon (10am-4pm, moderate traffic): $F_{\text{time}} = 0.95$
- Evening (5pm-7pm, peak return commute): $F_{\text{time}} = 0.85$ (15% reduction)
- Night (8pm-5am, reduced visibility, impaired drivers): $F_{\text{time}} = 0.60$ (40% reduction) [31]

Location Type Factor Multipliers:

- Highway/expressway: $F_{\text{location}} = 1.0$ (higher speeds permissible)
- Peri-urban/secondary road: $F_{\text{location}} = 0.85$ (15% reduction)
- Urban/local street: $F_{\text{location}} = 0.60$ (40% reduction)
- School zone: $F_{\text{location}} = 0.50$ (50% reduction)
- Construction/accident zone: $F_{\text{location}} = 0.40$ (60% reduction)

Visibility Factor Multipliers:

- Excellent visibility (>500m): $F_{\text{visibility}} = 1.0$

- Good visibility (200-500m): $F_{\text{visibility}} = 0.90$
- Fair visibility (100-200m): $F_{\text{visibility}} = 0.75$
- Poor visibility (50-100m, fog conditions): $F_{\text{visibility}} = 0.60$
- Very poor visibility (<50m, heavy fog/night): $F_{\text{visibility}} = 0.35$ [20]

Illustrative Example: Base speed limit 100 km/h on highway during rain at night in clear visibility:

$$\begin{aligned}
\text{Factors:} \quad F_{\text{weather}} &= 0.75 \text{ (rain)} \\
F_{\text{time}} &= 0.60 \text{ (night)} \\
F_{\text{location}} &= 1.0 \text{ (highway)} \\
F_{\text{visibility}} &= 0.80 \text{ (night, but clear conditions)} \\
W_{\text{total}} &= 0.40 + 0.20 + 0.20 + 0.20 = 1.0 \\
\text{RecommendedSpeed} &= 100 \times \frac{(0.40 \times 0.75) + (0.20 \times 0.60) + (0.20 \times 1.0) + (0.20 \times 0.80)}{1.0} \\
&= 100 \times \frac{0.30 + 0.12 + 0.20 + 0.16}{1.0} \\
&= 100 \times 0.78 = 78 \text{ km/h}
\end{aligned}$$

This algorithmic approach aligns with empirical research demonstrating that weather, time of day, location type, and visibility independently correlate with accident risk, and that combined consideration improves safety compared to single-factor approaches [9, 11].

3.3.4 Application Layer

The Application Layer implements the user-facing interface and functionality:

- **User Interface**: Real-time speedometer display showing current speed, recommended speed, and risk status (green/yellow/red indicators).
- **Alert System**: Notifications notify driver when current speed exceeds recommended speed, with severity-based alert types (visual indicator, vibration alert, audible chime).
- **Trip History**: Records completed trips with statistics (duration, distance, maximum speed, average driving score).
- **Route Recommendations**: Prototype interface for showing internal route recommendations and safety scores. Full navigation, live map display, and historical accident hotspot integration are future work.
- **Settings and Customization**: User-configurable options including alert sensitivity, notification preferences, data sharing preferences.
- **Safety Information**: Educational content on road safety, speed-accident relationships, and system usage guidance.

3.4 Tools, Technologies, and Development Environment

3.4.1 Mobile Application Framework

Flutter: The application frontend is developed using Flutter, Google's open-source mobile framework for cross-platform development (iOS, Android, web). Flutter selection is justified by: (1) single codebase supporting multiple platforms reducing development effort; (2) performance characteristics suitable for real-time GPS applications; (3) built-in state management (Provider package); (4) active community and extensive package ecosystem [15].

Dependencies:

- **Geolocator (v9.0+)**: Provides GPS location services with accuracy specification ±5-10 meters, supports configurable distance and time filtering for location updates, implements background location tracking [5].
- **Provider (v6.0+)**: State management package implementing Provider pattern for reactive data binding, enabling application state (current speed, alerts, trip data) changes to automatically reflect in UI [13].
- **http/dio**: Not included in the current dependency file; this would be required for future weather or backend API integration.
- **image_picker**: Enables image capture from device camera (for visibility estimation model).
- **permission_handler**: Manages runtime permission requests (location, camera, calendar) required by various features.
- **sqflite**: Not included in the current dependency file; it is a suitable future package for persistent trip storage.
- **google_maps_flutter**: Not included in the current dependency file; it is a possible future package for map visualization and road classification.

3.4.2 Backend Infrastructure

No backend server was implemented in the current prototype. The completed application focuses on mobile-side speed monitoring, contextual parameter selection, speed recommendation, alerts, trip models, and route recommendation logic. Firebase, Node.js/Express,

PostgreSQL, or similar backend infrastructure should therefore be treated as future work for large-scale deployment, analytics, account management, or synchronization.

3.4.3 External APIs

Weather API: OpenWeatherMap is a suitable future source of live weather data, but it is not integrated in the current source code. The prototype currently uses selected weather parameters.

Mapping Data: OpenStreetMap and/or Google Maps API may be used in a future version for road network data, location type classification, and map visualization. The current prototype uses internal route recommendation logic.

3.4.4 Development Tools

- **IDE**: Android Studio or Visual Studio Code with Flutter plugins for development and debugging
- **Version Control**: Git (GitHub/GitLab) for source code management
- **Testing Frameworks**: Flutter testing framework (unit tests), Mockito for mocking, integration tests for end-to-end scenarios
- **Future CI/CD**: GitHub Actions or a similar service can later be configured for automated testing on commits
- **Documentation**: Dart documentation tools, README files, API documentation

3.5 Data Collection and Testing Methodology

3.5.1 Testing Strategy

A multi-layered testing strategy ensures system quality:

3.5.1.1 Unit Testing

Each functional module (speed calculation, GPS processing, weather data handling, alert generation) is tested independently with predefined inputs and expected outputs. Unit tests verify:

- Speed calculation algorithm correctness against manual calculation verification
- Data preprocessing correctly removes outliers and handles missing values
- Alert generation triggers at correct speed thresholds
- Timestamp processing correctly identifies day periods

Target: >80% code coverage for core processing modules.

3.5.1.2 Integration Testing

Components are tested together to verify data flow between layers. Integration tests verify:

- GPS data correctly flows from Geolocator through data acquisition layer to processing layer
- Weather parameter selection integrates correctly with speed calculation algorithm

- Calculated speed recommendations correctly generate alerts when thresholds exceeded
- Historical data aggregation correctly computes driving statistics

3.5.1.3 System Testing

Complete application functionality is tested in simulated scenarios:

- Simulated GPS traces representing various road conditions (highway/urban, day/night, clear/rain)
- Weather condition variation testing to verify algorithm adjusts appropriately
- Performance testing measuring battery drain, memory consumption, processing latency
- Usability testing with representative users assessing interface clarity and alert notification effectiveness

3.5.1.4 Field Testing (if feasible)

Real-world testing on selected road segments validates system performance in actual driving conditions. Instrumented vehicle collects GPS traces, weather data, and perceived road safety observations for comparison against algorithm outputs.

3.5.2 Performance Metrics

System performance is evaluated through:

- **GPS Accuracy**: Comparison of device GPS location against ground truth (survey-grade GPS or known waypoints). Target: ±5-10 meters horizontal accuracy consistent with GPS specification.

- **Algorithm Accuracy**: Validation of recommended speeds against independent expert assessment (e.g., "algorithm recommends 60 km/h on highway in rain—is this reasonable?"). Percentage of algorithm outputs rated as appropriate/safe by safety experts.

- **Response Latency**: Time from condition change (weather update, location change) to updated recommended speed display. Target: <2 seconds.

- **Battery Consumption**: Measured battery drain during continuous GPS tracking over 8-hour driving day. Target: <15% additional battery drain compared to baseline (device without app running).

- **Data Accuracy**: Verification that trip data (distance, duration, speeds, alerts) correctly recorded and persisted.

- **User Experience**: Qualitative assessment of alert notification timing, interface usability, and driver acceptance of recommendations.

3.6 Ethical Considerations and Data Privacy

This project implements ethical practices and privacy protections:

- **Informed Consent**: Any driver testing involves explicit consent and understanding of data collection practices.

- **Data Minimization**: Application collects only necessary data (GPS location, speed, weather) and does not capture driver identity or vehicle identification unless explicitly opted-in by user.
- **Privacy by Design**: Trip data is stored locally on user device with no mandatory transmission to cloud. User retains control of data sharing preferences.
- **Anonymization**: Any aggregated data used for analysis or trend reporting is anonymized to prevent individual identification.
- **Regulatory Compliance**: System design aligns with Uganda's data protection framework and applicable international standards (General Data Protection Regulation if applicable).

3.7 Timeline and Milestones

The complete development timeline spans 12 weeks:

Phase	Key Activities	Timeline
1. Planning & Requirements	Define objectives, identify data sources, outline system design	Week 1-2
2. System Design & Architecture	Develop system architecture, design diagrams, technology selection	Week 3-4
3. Data Collection & Model Development	Implement weather model, collect sensor data, algorithm development	Week 5-6
4. System Implementation	Full system development, layer integration, core functionalities	Week 7-9
5. Testing & Evaluation	Comprehensive testing, performance evaluation, user acceptance testing	Week 10-11
6. Documentation & Presentation	Final report, user manuals, presentation preparation	Week 12

Table 3.2: Dynamic Speed Limit System Development Timeline

This methodology provides a structured, iterative, and empirically grounded approach to developing the Dynamic Speed Limit System. The combination of Design Science Research principles, Agile development practices, and rigorous testing methodology balances innovation with systematic evaluation of effectiveness.

Chapter 4

Data Representation, Analysis and Interpretation

4.1 Introduction

This chapter presents the results obtained from field and functional testing of the Dynamic Speed Limit System prototype. The purpose of the testing was to verify whether the application could display current speed, compute and show a recommended speed, alert the driver when the current speed exceeded the recommended speed, and present information in a way drivers could understand during use.

4.2 Testing Environment

Testing was carried out around Kihumuro, Mbarara–Kasese Road, Mile 4 Roundabout, and Mile Three Trading Centre. These locations were selected because they provided realistic road-use conditions including moving vehicles, roundabout traffic, and trading-centre activity. The tests focused on GPS speed response, recommended speed display, alert behaviour, and driver interpretation of the application's output.

Item	Description
Test locations	Kihumuro, Mbarara–Kasese Road, Mile 4 Roundabout, and Mile Three Trading Centre
Testing approach	Field observation and functional testing using the running mobile application
Main features tested	GPS speed display, recommended speed, over-speed warning, severe-weather warning message, and screen readability
Evidence collected	Application screenshots captured during testing
User feedback source	Informal feedback from drivers who interacted with or observed the application during testing

Table 4.1: Field testing environment

4.3 Functional Testing Results

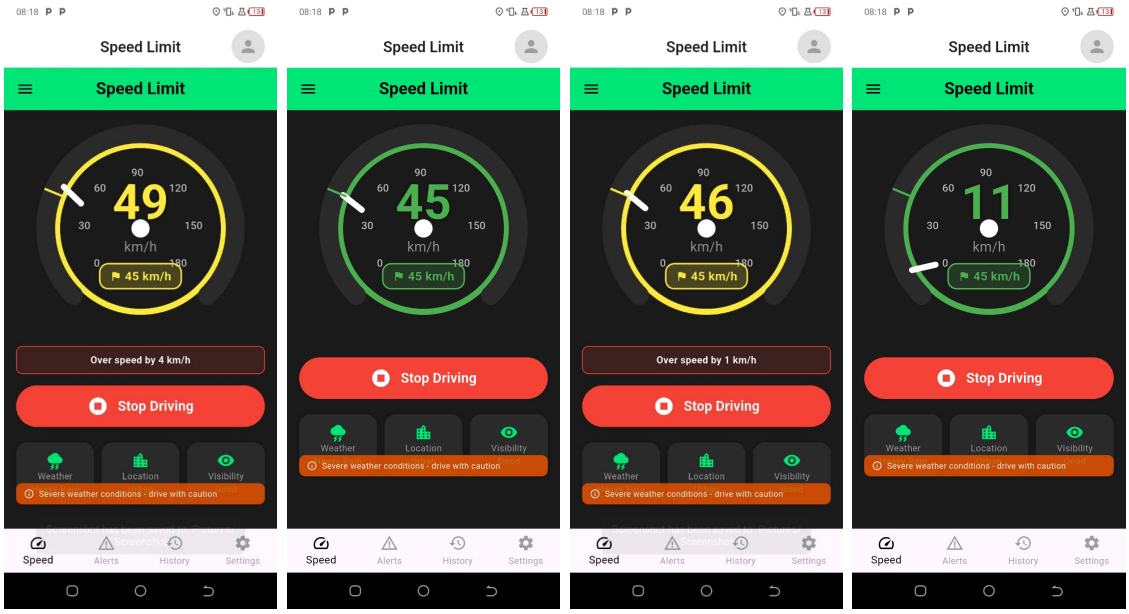
Table 4.2 summarizes the main functional tests carried out. The tests show that the application responded as expected by displaying the current speed, maintaining a recommended speed value, and showing over-speed warnings when the current speed exceeded the recommended speed.

Feature Tested	Expected Result	Observed Result	Status
Current speed display	Application should show vehicle speed in km/h during movement.	Screenshots showed changing speeds such as 11 km/h, 45 km/h, 46 km/h, and 49 km/h.	Pass
Recommended speed display	Application should display the recommended speed for the selected conditions.	The application displayed a recommended speed of 45 km/h during the test scenario.	Pass
Safe-speed condition	When current speed is equal to or below recommended speed, no over-speed warning should appear.	At 11 km/h and 45 km/h, the display remained in the safe state.	Pass
Over-speed condition	When current speed exceeds recommended speed, the app should show an over-speed warning.	At 46 km/h and 49 km/h against a 45 km/h recommendation, warnings of 1 km/h and 4 km/h over-speed were displayed.	Pass
Severe-weather warning	When severe weather is selected, the system should show a caution message.	The application displayed “Severe weather conditions - drive with caution.”	Pass
Driver readability	Drivers should be able to understand the displayed speed and warning messages.	Drivers reported that the speed value, recommended speed, and warning messages were understandable.	Pass

Table 4.2: Functional testing results

4.4 Screenshot Evidence

Figure 4.1 shows selected screenshots captured during testing. The screenshots demonstrate safe-speed and over-speed states in the application.



(a) Safe state at 11 km/h (b) Recommended speed at 45 km/h (c) 1 km/h over-speed warning (d) 4 km/h over-speed warning

Fig. 4.1: Application responses captured during field testing

4.5 Interpretation of Results

The results show that the prototype performed the main expected functions during field testing. The GPS speed display changed as the vehicle moved, indicating that the application was receiving and processing movement data. The recommended speed remained visible, allowing the driver to compare current speed with the safer suggested value. When current speed exceeded the recommended speed, the system displayed an over-speed warning, which confirms that the alert logic was functioning.

The screenshots also show that the interface was readable during testing. The large

speedometer display, colour changes, and direct over-speed message made the system output easy to interpret. Driver feedback was positive; the drivers were able to understand the application features and the meaning of the results being displayed.

4.6 Limitations Observed During Testing

Some limitations were observed. First, the weather support was not fully reliable because the available weather API/service did not always provide accurate localized weather readings for the exact test environment. This means weather-based recommendations should be improved with a stronger weather service or additional validation. Second, GPS readings can vary depending on signal strength, road environment, and device quality, especially near busy trading centres or roundabouts. Third, the user feedback was informal and based on a small group of drivers, so wider usability testing is still required. Finally, the testing verified prototype behaviour but did not measure long-term accident reduction, which would require extended real-world deployment.

4.7 Chapter Summary

Chapter 4 presented the testing environment, functional testing results, screenshot evidence, interpretation, and observed limitations. Overall, the prototype demonstrated the core expected behaviour: it displayed current speed, showed a recommended speed, generated over-speed warnings, and was understandable to drivers during field testing.

Chapter 5

System Development and Implementation

5.1 Introduction

This chapter describes the complete system development and implementation of the Dynamic Speed Limit System. The chapter details identified requirements (user, functional, non-functional), system architecture, technology platform (Flutter), core components, user interface design, system specifications, and testing methodology. The system is implemented as a cross-platform mobile application with real-time GPS integration, weather data APIs, and local data persistence supporting dynamic speed recommendations based on environmental conditions.

5.2 Requirements Analysis and Specification

5.2.1 User Requirements

User requirements capture what end-users and stakeholders need from the system:

1. **Real-Time Speed Monitoring**: Drivers require continuous, real-time display of their current vehicle speed with minimal latency (<1 second update interval).

2. **Speed Recommendations**: Drivers need clear, easy-to-understand recommendations for safe driving speeds based on current road and environmental conditions.

3. **Visual Safety Indicators**: Drivers require immediate visual feedback (color-coded indicators) showing whether their current speed is safe, moderately safe, or dangerous relative to conditions.

4. **Alert Notifications**: Drivers need timely alerts when their speed exceeds recommended limits, with multiple notification modalities (visual, audible, haptic) allowing driver customization.

5. **Trip Documentation**: Drivers want records of their trips including duration, distance traveled, average/maximum speeds, and driving safety scores for performance tracking.

6. **Ease of Use**: The application must be intuitive and usable while driving, with simple parameter adjustment and minimal cognitive load.

7. **Accessibility**: The system must function on standard smartphones available in Uganda and developing regions, requiring no specialized hardware beyond built-in GPS and internet connectivity.

8. **Educational Content**: Drivers desire information about road safety principles, the relationship between speed and accident risk, and best practices for safe driving.

9. **Customization**: Users want configurable preferences for alert sensitivity, notification types, data sharing, and system behavior.

10. **No Cost Barrier**: The application should be freely available or low-cost, accessible to drivers across economic strata.

5.2.2 Functional Requirements

Functional requirements specify what the system must do:

1. **GPS Speed Acquisition** (FR-1): The system shall continuously acquire vehicle location via GPS at configurable intervals (5-meter distance filter default), calculate instantaneous speed from GPS velocity data, and display speed in km/h with ±0.18 km/h accuracy [36].

2. **Weather Data Integration** (FR-2): The system shall integrate real-time weather data from external APIs, retrieving current weather condition (clear, rain, fog, snow, storm), precipitation amount, visibility, and wind speed with update frequency configurable from 30-60 seconds.

3. **Environmental Parameter Detection** (FR-3): The system shall automatically determine or allow user input for: (a) time of day (morning, afternoon, evening, night) from device clock, (b) location type (highway, urban, residential, school zone, construction zone), (c) visibility level (excellent to very poor) from light sensor and time-based estimation.

4. **Speed Calculation Algorithm** (FR-4): The system shall implement multi-factor speed calculation algorithm (detailed in Chapter 3, Section 3.3.3.1) computing recommended safe speed from base speed limit, weather condition, time of day, location type, and visibility using weighted averaging of four factors.

5. **Alert Generation** (FR-5): The system shall generate safety alerts when current vehicle speed exceeds recommended speed by threshold amount (configurable, default 5 km/h), classifying alerts by severity level (low, medium, high) based on speed exceedance magnitude.

6. **Alert Notifications** (FR-6): The system shall deliver alerts to driver via multiple channels: (a) on-screen visual indicator change (green→yellow→red), (b) optional audible chime notification, (c) optional haptic vibration feedback, with user control over active notification modalities.

7. **Trip Tracking** (FR-7): The system shall record ongoing trips including: start time, current/maximum/average speeds, location type, number of speed-limit exceedances, alerts generated, trip duration, distance traveled. Trip data shall persist on device storage.

8. **Driving Score Calculation** (FR-8): The system shall compute driving safety score for each trip based on: (a) percentage of trip at recommended speed, (b) number of speed exceedances, (c) maximum speed exceedance magnitude, (d) alert frequency, producing single numerical score 0-100 for driver feedback.

9. **Trip History Management** (FR-9): The system shall display past trip records with full statistics, support filtering/sorting by date/location/score, allow trip detail inspection, and support data export functionality.

10. **Route Recommendations** (FR-10): The system shall provide a prototype interface for drivers to view internal route recommendations and safety scores. Turn-by-turn navigation and live accident hotspot display are future work.

11. **User Settings Management** (FR-11): The system shall provide settings interface allowing configuration of: alert sensitivity level, active notification types, data retention policies, location type preferences, display themes, language selection.

12. **Educational Content Delivery** (FR-12): The system shall provide in-app educational screens explaining: road safety statistics, speed-accident relationships, system functionality ("How It Works"), safety disclaimer and legal notices, developer

information.

13. **User Feedback Mechanism** (FR-13): The system shall provide contact interface allowing users to submit feedback, report bugs, ask questions, with automatic connection to development team contact information.

14. **Persistent Data Storage** (FR-14): The system shall support storage of trip data, user settings, and alert history. The current prototype includes trip models and JSON serialization support, while SQLite persistence remains future work.

15. **Offline Functionality** (FR-15): The system shall continue core functionality (GPS tracking, speed calculation using cached parameters, local alert generation) during temporary internet connectivity loss, synchronizing cached data upon reconnection.

16. **Permission Management** (FR-16): The system shall properly request and manage Android/iOS runtime permissions for: location services, camera access (for visibility sensing), calendar access (for scheduling), notification permissions, displaying clear explanations of required permissions.

17. **Multi-Platform Support** (FR-17): The system shall be deployable on both Android (minimum API 21) and iOS (minimum iOS 11.0) platforms through single Flutter codebase.

18. **Background Operation** (FR-18): The system shall support background GPS tracking while application is minimized, with configurable background activity duration limit for battery preservation.

5.2.3 Non-Functional Requirements

Non-functional requirements specify system quality attributes:

5.2.3.1 Performance Requirements

- **GPS Update Latency**: GPS position updates shall occur within 0.5-1 second of occurrence, speed values shall update within 1 second of velocity change.
- **Speed Calculation Latency**: Multi-factor speed calculation shall complete within 200 milliseconds of input data availability, with recommended speed displayed within 500ms of calculation trigger event.
- **Future Weather API Response Time**: When live weather integration is added, weather data retrieval should complete within 5 seconds, with graceful fallback to cached data upon API timeout.
- **Alert Generation Response**: Safety alert shall generate and display to user within 1 second of speed threshold exceedance.
- **Battery Consumption**: Continuous GPS tracking, processing, and display shall not exceed 15% battery drain per 8-hour driving day (approximately 1.875% per hour), recognizing that this represents additional consumption beyond baseline device usage [8].
- **Memory Efficiency**: Application memory footprint shall not exceed 200 MB during normal operation, supporting deployment on devices with minimum 2 GB available RAM.
- **Database Performance**: Trip history query operations (retrieve past trips, compute statistics) shall complete within 2 seconds even with 1,000+ historical trips stored.
- **Scalability**: Future backend deployment should be designed to support many concurrent users. The current prototype was evaluated as a standalone mobile application, not as a cloud-scale service.

5.2.3.2 Reliability and Availability

- **System Uptime**: Mobile application shall achieve 99% uptime excluding planned maintenance, recovering gracefully from transient errors.
- **GPS Failure Recovery**: Upon GPS signal loss, system shall automatically attempt reconnection within 30 seconds, displaying clear status to user, and resuming tracking upon recovery.
- **Data Persistence**: Trip data shall be reliably stored and not lost due to application crash or device restart, protecting driver records.
- **Error Handling**: All error conditions shall be handled gracefully with user-friendly error messages and automatic recovery attempts where applicable.

5.2.3.3 Security Requirements

- **Data Protection**: User location data shall be protected from unauthorized access; transmission to servers shall use industry-standard encryption (TLS 1.2+).
- **Privacy Preservation**: Application shall not transmit personally-identifiable information without explicit user consent; trip data shall be anonymized in any aggregated analysis.
- **Input Validation**: All user input and external API data shall be validated to prevent injection attacks or malformed data processing.
- **Secure Storage**: Cached authentication credentials and sensitive user settings shall be stored using platform-provided secure storage (Android Keystore, iOS Keychain).

5.2.3.4 Usability Requirements

- **Intuitive Interface**: User interface shall be learnable without documentation, with standard UI patterns and clear visual hierarchy.
- **Accessibility**: Application shall support text scaling and high-contrast display modes; critical alerts shall not rely solely on color differentiation.
- **Localization**: Application shall support multiple languages (minimum English + local languages) with proper text handling and regional settings.
- **Mobile-Optimized**: UI shall be fully responsive to various screen sizes (4-7 inches typical mobile range), with touch-friendly button/control sizing (minimum 48 dp touch targets).

5.2.3.5 Scalability and Maintainability

- **Modular Architecture**: Code shall be organized into logically separate modules (data acquisition, processing, UI, services) supporting independent development and testing.
- **Code Reusability**: Common functionality shall be abstracted into reusable components and utilities reducing code duplication.
- **Documentation**: Code shall be documented with inline comments, JSDoc notation, and separate technical documentation enabling future maintenance.
- **Backward Compatibility**: New versions shall maintain backward compatibility with existing stored data and settings, supporting seamless upgrade paths.

5.3 System Architecture and Technical Design

5.3.1 Layered Architecture Overview

The Dynamic Speed Limit System is implemented using a layered architecture pattern separating functional concerns:

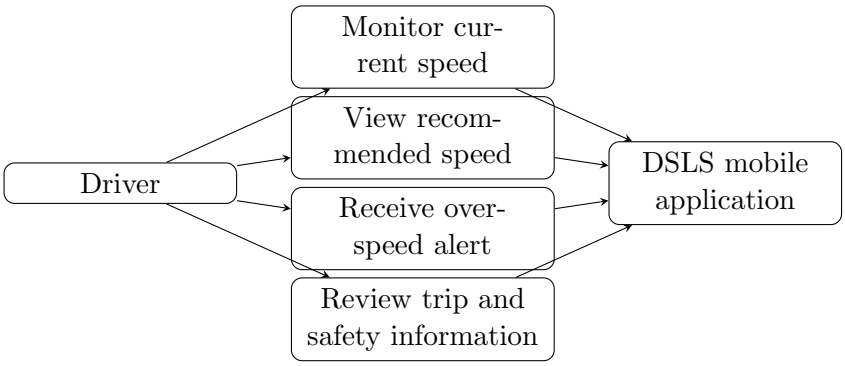


Fig. 5.1: Use case representation of the Dynamic Speed Limit System

5.3.1.1 Presentation Layer (UI)

The Presentation Layer implements all user-facing interface and interactions:

- **Speed Monitoring Screen**: Real-time speedometer with large, easy-to-read speed display, recommended speed indicator, color-coded risk status (green/yellow/red), current conditions display (weather, time, location type, visibility).
- **Dashboard Screen**: Overview of current trip status, cumulative statistics, quick access to main functions.
- **Trip History Screen**: Scrollable list of past trips with summary information (date, duration, distance, driving score), filtering and sorting options, drill-down to trip details.

- **Route Recommendation Screen**: Displays internal route recommendations, estimated time, distance, route type, and safety score. Live map visualization, accident hotspot indicators, and turn-by-turn navigation are future work.
- **Alerts Screen**: Historical record of safety alerts generated during trips, with timestamp, speed values, conditions when alert triggered.
- **Settings Screen**: User preferences interface for alert sensitivity, notification modalities, display options, data management.
- **Educational Screens**: "How It Works", "Safety Disclaimer", "Developers", "Contact Us" information pages.
- **Navigation Structure**: Tab-based bottom navigation (Flutter) providing access to primary screens, with drawer menu for secondary functions.

5.3.1.2 Services/Business Logic Layer

The Services Layer implements core application logic and state management:

- **GpsSpeedService**: Manages GPS positioning, speed calculation, handles GPS permissions, initializes tracking, directs position stream to UI updates.
- **SpeedAlertService**: Generates speed alerts when thresholds exceeded, manages alert history, handles notification delivery to user.
- **RouteService**: Manages prototype route recommendations using internal route examples and safety score logic. Accident data integration and navigation are not completed in the current source.
- **TripService**: Records ongoing trip data, computes trip statistics (duration, distance, avg/max speeds), calculates driving score, manages trip persistence.

- **State Management (Provider)**: Implements reactive state management using Provider pattern; UI widgets rebuild automatically when underlying data (speed, alerts, trip info) changes.

5.3.1.3 Data Processing Layer

The Data Processing Layer handles calculation and data transformation:

- **Speed Calculation Module**: Implements SpeedCalculator class computing recommended speed from SpeedParameters (weather, time, location, visibility, base limit) using multi-factor weighted algorithm.
- **Parameter Determination**: Determines or accepts parameter values from available application inputs. The current prototype supports contextual parameter selection; live weather API input and map-based road classification are future work.
- **Data Validation**: Handles basic permission and tracking errors. GPS outlier filtering and weather API validation are future improvements.
- **Aggregation**: Computes trip statistics (trip duration, distance, speed statistics, alert counts) from raw GPS and event data.

5.3.1.4 Storage Layer

The Storage Layer manages data persistence:

- **Local Data Model**: Represents trip records, alerts, speed values, and user-related settings in Dart model/service classes. SQLite persistence is planned for a future version.

- **Data Models**: TripData class (encapsulating trip information), SpeedAlert class (individual alert records), SpeedParameters (current environmental state), configuration objects.

- **Query Interface**: Database access through abstracted interface supporting: insert trip, retrieve trip by ID, retrieve all trips with filtering, update trip data, delete trip record, transaction support for data consistency.

5.3.1.5 External Integration Layer

The External Integration Layer connects to external systems:

- **GPS Hardware Integration**: Via Geolocator package, provides LocationAccuracy.high mode, 5-meter distance filter, continuous position stream.

- **Future Weather API Integration**: OpenWeatherMap API calls may later retrieve real-time weather for current coordinates. This is not completed in the current source.

- **Future Map Data Integration**: OpenStreetMap or Google Maps API may later support map visualization and location type classification.

- **Future Cloud Backend**: Firebase or a comparable backend may later support trip aggregation, historical analysis, and system performance monitoring, but no cloud backend is implemented in the current source.

5.3.2 Data Flow Overview

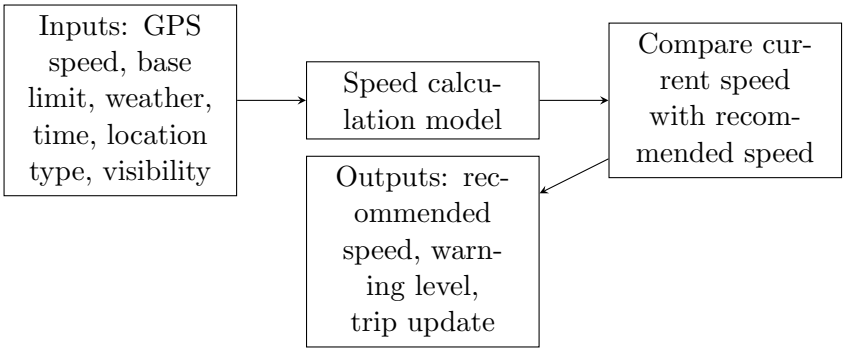


Fig. 5.2: Data flow from contextual input and GPS speed monitoring to speed recommendation and alert output

The primary data flow operates as follows:

1. GPS hardware continuously provides position updates to GpsSpeedService (every 0.5-1 second)

2. GpsSpeedService calculates velocity from position delta and time delta, exposes currentSpeed property

3. Speed Monitoring Screen subscribes to currentSpeed via Provider; UI rebuilds with updated value

4. Weather service queries API at configurable interval, caches weather condition

5. Trip service aggregates GPS positions and speeds into persistent storage

6. When user selects speed calculation parameters (weather, time, location, visibility) or when automatic parameter detection changes, SpeedCalculator.calculate() runs with current parameters

7. Algorithm outputs recommendedSpeed; if currentSpeed > recommendedSpeed + threshold, SpeedAlertService generates alert

8. Alert triggers notification (visual, audible, haptic); alert record persists to database
9. UI reflects updated state (color change to red, alert notification, alert counter increment)
10. Upon trip completion, TripService computes aggregate statistics (duration, distance, avg/max speed, alert count, driving score) and persists to database

5.4 Mobile Application Implementation

5.4.1 Technology Platform: Flutter

The system is implemented using Flutter, Google's open-source mobile development framework selected for the following characteristics:

- **Cross-Platform Development**: Single Dart codebase compiles to native Android and iOS applications, eliminating duplicate development effort compared to separate native implementation.
- **Performance**: Compiled native code execution provides performance comparable to native development, essential for real-time GPS processing and frequent screen updates.
- **Hot Reload**: Supports iterative development with instant code reload during development, accelerating development cycle.
- **Rich Widget Library**: Comprehensive built-in UI widgets (Material Design) enable rapid interface development without external UI framework dependencies.
- **State Management**: Provider package enables reactive state management with automatic UI rebuilding when data changes, simplifying complex state handling.

- **Package Ecosystem**: Extensive third-party package availability (pub.dev) provides GPS, permissions, local storage, HTTP, and other required functionality without reimplementation.
- **Community and Support**: Large active community provides tutorials, examples, and rapid issue resolution.

5.4.2 Key Dependencies and Packages

The project pubspec.yaml specifies the following key dependencies (versions as of implementation):

1. **Flutter SDK**: Version $^{3}.7.2, providingcoreframework, widgets, andruntime.$

2. **Geolocator (v13.0.2)**: Provides GPS/location services interface, requesting current position or continuous position stream. Configuration:
    - LocationAccuracy.high: Best accuracy mode, utilizes GPS + assisted positioning
    - distanceFilter: 5 meters—only generate position update when device moves >5 meters
    - Timeout: 10 seconds default for position requests

    GPS accuracy specification: ±5-10 meters horizontal, ±0.18 km/h velocity [36]

3. **Provider (v6.1.2)**: State management package implementing Provider architectural pattern. Key usage:
    - ChangeNotifierProvider: Wraps services (GpsSpeedService, TripService) providing reactive updates

- Consumer widget: Subscribes UI to service changes, rebuilding only affected widgets
- MultiProvider: Initializes multiple providers at app startup

4. **image_picker (v1.2.1)**: Enables camera access for light sensor visualization and light-based visibility estimation.

5. **permission_handler (v11.4.0)**: Runtime permission management for Android/iOS:
   - Location permissions: Required for GPS tracking
   - Camera permissions: Optional, for visibility estimation
   - Notification permissions: Required for alert delivery

6. **cupertino_icons (v1.0.8)**: iOS-style icon library for platform consistency.

7. **flutter_launcher_icons (v0.13.1)**: Tool for generating app launcher icons across platforms from single source image.

5.4.3 Core Components and Models

5.4.3.1 Speed Calculator Module

Located in `lib/models/speed_calculator.dart`, implements the multi-factor speed calculation algorithm:

Enumerations (Type Safety):

- WeatherCondition: clear, cloudy, rain, heavyRain, fog, snow, storm
- DayPeriod: morning, afternoon, evening, night

- LocationType: urban, suburban, highway, residential, schoolZone, constructionZone
- VisibilityLevel: excellent, good, moderate, poor, veryPoor

SpeedParameters Class (immutable data container):

```
class SpeedParameters {
  final WeatherCondition weather;
  final DayPeriod timeOfDay;
  final LocationType location;
  final VisibilityLevel visibility;
  final int baseSpeedLimit;
}
```

Contains all inputs required for speed calculation; provides factory constructor for default values.

SpeedCalculationResult Class:

```
class SpeedCalculationResult {
  final int recommendedSpeed;
  final int riskLevel; // 0-100
  final String riskDescription; // LOW/MEDIUM/HIGH
  final List<String> warnings;
}
```

Encapsulates calculation output; provides risk level assessment and applicable warnings.

SpeedCalculator Utility: Implements static methods computing multiplier for each factor:

- _getWeatherMultiplier(WeatherCondition): Returns 0-100 percentage multiplier (e.g., rain returns 75, meaning 25% speed reduction)
- _getTimeMultiplier(DayPeriod): Returns time-of-day multiplier (night returns 60, indicating 40% speed reduction)
- _getLocationMultiplier(LocationType): Returns location-specific multiplier (schoolZone returns 30, indicating 70% speed reduction)
- _getVisibilityMultiplier(VisibilityLevel): Returns visibility-based multiplier (veryPoor returns 35)

Calculation Algorithm:

```
static SpeedCalculationResult calculate(SpeedParameters params) {
  // Individual factor multipliers (0-100 percentage)
  final weatherMult = _getWeatherMultiplier(params.weather);
  final timeMult = _getTimeMultiplier(params.timeOfDay);
  final locationMult = _getLocationMultiplier(params.location);
  final visibilityMult = _getVisibilityMultiplier(params.visibility);

  // Average of four factors
  final overallMultiplier = (weatherMult + timeMult +
    locationMult + visibilityMult) / 400;

  // Apply to base speed and clamp to valid range
  int recommendedSpeed =
    (params.baseSpeedLimit * overallMultiplier).round()
    .clamp(minSpeed: 20, maxSpeed: 130);
```

```
  // Compute risk score (inverse of multiplier)
  final riskScore = 100 - ((overallMultiplier - 0.3) * 100).round();
  final riskLevel = riskScore.clamp(0, 100);

  // Generate risk description
  String riskDescription = riskLevel >= 70 ? 'HIGH' :
    riskLevel >= 40 ? 'MEDIUM' : 'LOW';

  // Collect applicable warnings
  List<String> warnings = [...];

  return SpeedCalculationResult(
    recommendedSpeed: recommendedSpeed,
    riskLevel: riskLevel,
    riskDescription: riskDescription,
    warnings: warnings,
  );
}
```

5.4.3.2 Trip Data Models

Located in `lib/models/trip_data.dart`:

TripData Class:

```
class TripData {
  final String id;
```

```
  final DateTime startTime;
  final DateTime? endTime;
  final LocationType location;
  final int baseSpeedLimit;
  final int recommendedSpeed;
  final int maxSpeed;
  final int avgSpeed;
  final int overSpeedCount;
  final List<SpeedAlert> alerts;

  // Computed properties
  Duration? get duration => endTime?.difference(startTime);
  String get formattedDuration => "2h 15m";
}
```

Represents a single trip with all relevant statistics. JSON serialization support enables database persistence [13].

SpeedAlert Class: Represents individual alert event occurrence during trip with timestamp, speed values, and alert type.

5.4.4 Service Layer Implementation

5.4.4.1 GpsSpeedService

Located in `lib/services/gps_speed_service.dart`, manages GPS position acquisition and speed calculation:

Properties:

- _positionStream: Subscription to continual position updates from Geolocator
- _currentSpeed: Last calculated speed (km/h)
- _isTracking: Boolean flag indicating GPS tracking active
- _hasPermission: Boolean tracking location permission status
- _lastPosition: Most recent GPS position received
- _lastUpdate: Timestamp of last position update

Key Methods:

checkPermission(): Verifies location services enabled and requests runtime permissions if needed:

```
Future<bool> checkPermission() async {
  // Check location services enabled
  bool serviceEnabled =
    await Geolocator.isLocationServiceEnabled();
  if (!serviceEnabled) return false;

  // Check and request permissions
  LocationPermission permission =
    await Geolocator.checkPermission();
  if (permission == LocationPermission.denied) {
    permission = await Geolocator.requestPermission();
  }

  return permission == LocationPermission.whileInUse ||
```

```
    permission == LocationPermission.always;
}
```

startTracking(): Initiates continuous GPS position stream:

```
Future<void> startTracking() async {
  if (_isTracking) return;

  final hasPerms = await checkPermission();
  if (!hasPerms) return;

  const locationSettings = LocationSettings(
    accuracy: LocationAccuracy.high,
    distanceFilter: 5, // Update every 5 meters
  );

  _positionStream =
    Geolocator.getPositionStream(
      locationSettings: locationSettings
    ).listen(_onPositionUpdate);

  _isTracking = true;
  notifyListeners();
}
```

_onPositionUpdate(Position position): Callback invoked when GPS position received:

```
void _onPositionUpdate(Position position) {
```

```
  if (_lastPosition != null && _lastUpdate != null) {
    // Calculate speed from position delta
    final distance = _lastPosition!
      .distanceTo(position);  // meters
    final timeDiff =
      position.timestamp!.difference(_lastUpdate!)
      .inMilliseconds / 1000;  // seconds

    _currentSpeed =
      (distance / timeDiff * 3.6)  // m/s to km/h
      .toInt();

    notifyListeners();  // Trigger UI rebuild
  }

  _lastPosition = position;
  _lastUpdate = position.timestamp;
}
```

stopTracking(): Halts position stream and GPS tracking.

5.4.4.2 SpeedAlertService

Located in `lib/services/alert_service.dart`, manages alert generation and notification:

- Monitors speed exceedance conditions

- Maintains alert threshold (default 5 km/h above recommended)
- Generates SpeedAlert objects when thresholds exceeded
- Manages alert history
- Delivers notifications to UI and user via multiple channels (visual, audible, haptic)
- Configurable alert sensitivity level

5.4.4.3 TripService

Located in `lib/services/trip_service.dart`, records and manages trip data:

- Initiates trip recording at tracking start
- Aggregates GPS position and alert data throughout trip
- Computes trip statistics upon completion (duration, distance, speed statistics, driving score)
- Maintains trip records in the application service and supports JSON serialization; SQLite persistence is planned for future work
- Retrieves historical trips for Trip History screen
- Supports trip filtering and sorting

5.4.4.4 RouteService

Located in `lib/services/route_service.dart`, manages prototype route recommendations:

- Maintains internal route examples with distance, estimated time, route type, and safety score
- Adjusts route safety score according to current location type, weather condition, and time of day
- Demonstrates the intended route-safety concept without live mapping API integration
- Does not yet provide turn-by-turn navigation or verified accident hotspot data

5.4.5 User Interface Implementation

The application implements a tab-based navigation interface with drawer menu:

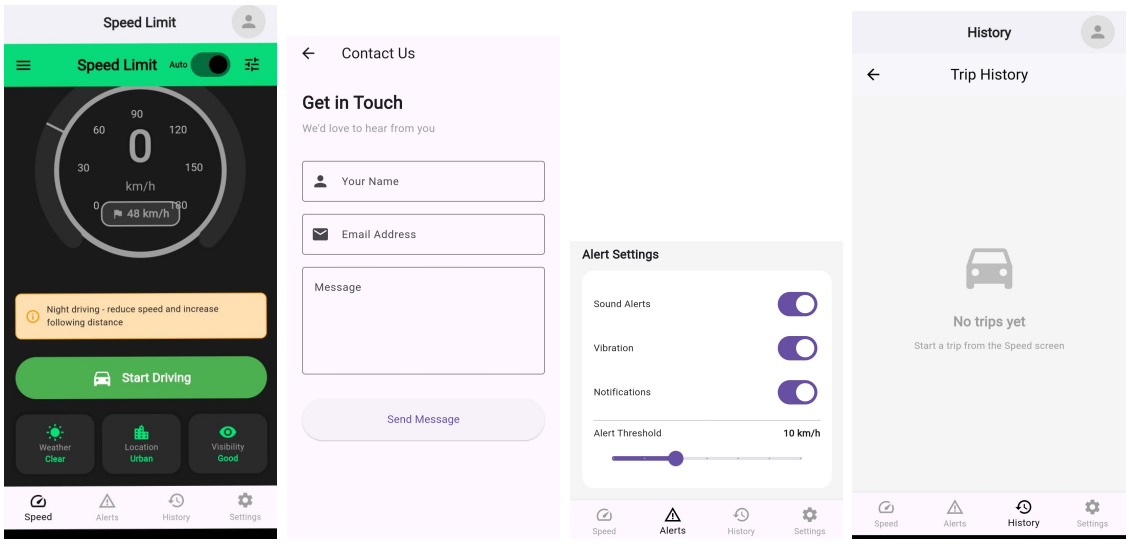


Fig. 5.3: Selected application screens from the Dynamic Speed Limit System prototype

5.4.5.1 Speed Monitoring Screen (speed_screen.dart)

Core interface displaying real-time speed information:

Visual Elements:

- Large numerical speed display (current speed in km/h) with font size >72pt for visibility while driving
- Recommended speed display with smaller font
- Color-coded risk indicator background (green recommended, yellow recommended, red recommended)
- Analog speedometer gauge visualization
- Current environmental conditions display (weather, time period, location type, visibility)
- GPS tracking status indicator

Interactions:

- Parameter picker bottom sheet allowing manual override of environmental parameters (for testing, user customization)
- Start/stop tracking toggle
- Navigation to other screens via bottom tab bar

5.4.5.2 Trip History Screen (history_screen.dart)

Displays past trip records with summary information:

Features:

- Scrollable list of trip records with: date/time, duration, distance, max speed, driving score
- Tap to view trip detail screen with full statistics, alert timeline, speed graph

- Filter options (by date range, location type, score range)
- Sort options (by date, score, distance)
- Export functionality (save trip data as CSV/JSON)

5.4.5.3 Alerts Screen (alerts_screen.dart)

Historical record of safety alerts:

Features:

- Chronological list of alerts with timestamp, speeds (current vs recommended), conditions
- Grouping by trip or day
- Export alert history

5.4.5.4 Settings Screen (settings_screen.dart)

User preferences and customization:

Options:

- Alert sensitivity slider (low/medium/high)
- Active notification types checkboxes (visual, audible, haptic)
- Display theme selection (light/dark mode)
- Data retention policy (how many historical trips to maintain)
- Location type auto-detection toggle

- Language selection
- About/help links

5.4.5.5 Educational Screens

Informational pages providing driver education:

- **How It Works** (how_it_works_page.dart): Explains system algorithm, data sources, recommendations generation
- **Safety Disclaimer** (safety_disclaimer_page.dart): Legal notices, system limitations, recommendations not legal requirement
- **Developers** (developers_page.dart): Project team information, university affiliation, acknowledgements
- **Contact Us** (contact_us_page.dart): Email contact, feedback form, bug reporting mechanism

5.5 System Requirements and Specifications

5.5.1 Hardware Requirements

The system is designed for deployment on standard modern smartphones with minimal hardware requirements:

Component	Minimum Specification
Processor	ARM Cortex-A53 1.4 GHz (typical 2016+ smartphones)
RAM Memory	2 GB (minimum for smooth operation)
Storage Space	150 MB available for application and trip history
GPS Hardware	Standard GNSS receiver (all modern smartphones)
Display	4.5” touchscreen display (1920x1080 minimum resolution)
Battery	2,500 mAh minimum (supports 8 hour driving day)
Network	Wi-Fi or mobile data connection (4G LTE recommended)

Table 5.1: Hardware Specifications - Dynamic Speed Limit System

Recommended Devices (typical modern smartphones):

- Android: Samsung Galaxy A-series, Xiaomi Redmi, Tecno Spark, Infinix Note
- iOS: iPhone 6S or newer
- Budget-Conscious Option: Android devices USD 100-150 (2-3 GB RAM, MediaTek processor)

5.5.2 Software Requirements

Component	Specification
Mobile OS - Android	Android 5.0 (API 21) or higher
Mobile OS - iOS	iOS 11.0 or higher
Local Database	SQLite 3 (included in OS)
Runtime Environment	Flutter Runtime 3.7.2+, Dart VM
Network Stack	HTTP/HTTPS for API communication
Location Services	Device GPS/GNSS provider, Android Location Manager, iOS CoreLocation
Permissions	Manifest permissions for location, camera, notifications

Table 5.2: Software Specifications - Dynamic Speed Limit System

5.5.3 External Dependencies and Future APIs

5.5.3.1 Weather Data API (Future Work)

Service: OpenWeatherMap (https://openweathermap.org/api)

Endpoint: Current weather data endpoint

Request Format:

```
GET /data/2.5/weather?lat={latitude}&lon={longitude}
  &appid={API_KEY}&units=metric
```

Response: JSON containing current_condition, temperature, precipitation, wind_speed, visibility

Planned Update Frequency: 30-60 second intervals (configurable)

Cost: Free tier: 1,000 calls/day; Commercial: USD 40-400/month depending on volume

Planned Fallback Behavior: Upon API unavailability, a future version should use cached weather data from the last successful query.

5.5.3.2 Mapping and Location APIs (Future Work)

Services: OpenStreetMap (free), Google Maps Platform (commercial)

Usage:

- Road network data: identify route type (highway/urban/residential)
- Map visualization: route display, landmark annotation
- Geocoding: convert GPS coordinates to address/location name

Planned Integration: A future version may use a map package for visualization and separate HTTP calls for road classification data.

5.5.4 Deployment and Installation

5.5.4.1 Prototype Deployment Status

The current project output is a Flutter prototype, not a published app-store release. The application can be run from the Flutter development environment on a compatible Android device or emulator after dependencies are installed. A production Android APK/AAB or iOS IPA release would require release signing, store assets, privacy policy preparation, and further field testing. These deployment activities are therefore treated as future work rather than completed implementation.

Deployment item	Status in this project
Flutter debug/prototype execution	Supported by the source project structure
Android release APK/AAB	Future release activity
iOS IPA/TestFlight release	Future release activity
Google Play or App Store publication	Not completed in this project
Privacy policy and public release documentation	Required before public deployment

Table 5.3: Prototype deployment status

5.6 Database Schema and Data Persistence

5.6.1 Proposed SQLite Database Design

The current source code includes trip and alert models with JSON serialization support. SQLite persistence is not yet implemented. The following schema is proposed for a future persistent storage layer:

5.6.1.1 Trips Table

```
CREATE TABLE trips (
  id TEXT PRIMARY KEY,
  start_time TEXT NOT NULL,
  end_time TEXT,
  location_type INTEGER,
  base_speed_limit INTEGER,
  recommended_speed INTEGER,
  max_speed INTEGER,
  avg_speed INTEGER,
  overspeed_count INTEGER,
```

```
  driving_score INTEGER,
  created_at TEXT DEFAULT CURRENT_TIMESTAMP
);
```

5.6.1.2 Alerts Table

```
CREATE TABLE alerts (
  id TEXT PRIMARY KEY,
  trip_id TEXT NOT NULL,
  alert_time TEXT NOT NULL,
  current_speed INTEGER,
  recommended_speed INTEGER,
  alert_type TEXT,
  FOREIGN KEY (trip_id) REFERENCES trips(id)
);
```

5.6.1.3 User Settings Table

```
CREATE TABLE settings (
  key TEXT PRIMARY KEY,
  value TEXT,
  updated_at TEXT DEFAULT CURRENT_TIMESTAMP
);
```

Key settings: alert_sensitivity, enable_sound, enable_vibration, theme_mode, language, etc.

5.6.2 Data Serialization

Complex objects (SpeedParameters, SpeedCalculationResult) serialize to/from JSON for database storage using Dart's jsonEncode/jsonDecode functions, enabling flexible schema evolution.

5.7 Testing and Quality Assurance

5.7.1 Testing Strategy

5.7.1.1 Test Environment and Evidence Sources

Testing evidence was documented using source-code inspection, controlled parameter scenarios, alert threshold checks, GPS permission behaviour, and review of the application screenshots shown in Figure 5.3. The exact physical device model and Android version used during screenshot capture were not recorded in the supplied project materials; this is a documentation gap that should be completed from the tester's phone settings before final submission.

Item	Recorded detail
Application framework	Flutter, Dart SDK environment
Main device capability tested	GPS/location permission and GPS-based speed calculation through Geolocator
Device used	Android smartphone used for prototype screenshots; exact model not recorded in supplied materials
Android version	Not recorded in supplied materials; should be inserted before final submission
Screenshot evidence	Speed, settings, route/history, and related screens shown in Figure 5.3
Usability participants	Count not recorded in supplied materials; exact participant number should be inserted from the project team's user testing notes

Table 5.4: Testing environment and evidence record

5.7.1.2 Unit Testing

Individual modules tested with predefined inputs and expected outputs:

- SpeedCalculator algorithm verification: test all multiplier combinations, verify output bounds checking
- TripService calculations: validate duration, distance, statistics computation
- Parameter determination logic: verify correct enum value assignment from sensor inputs

Target coverage for future automated testing is greater than 80% for core calculation modules. During refinement, additional unit test files were prepared for the SpeedCalculator and SpeedAlertService modules to formalize the expected behaviour of speed recommendation and alert thresholds.

5.7.1.3 Controlled Functional Test Cases

Table 5.5 presents concrete test cases used to verify the core behaviour of the prototype. The actual results are derived from the implemented multiplier values and alert thresholds in the source code.

Test Case	Input/Condition	Expected Result	Actual Result	Status
Normal urban condition	Clear weather, afternoon, urban, excellent visibility, base speed 60 km/h	Recommended speed close to base speed but adjusted for urban context	Recommended speed = 56 km/h; risk = LOW	Pass
Rain and night driving	Rain, night, highway, good visibility, base speed 100 km/h	Recommended speed reduced and warnings displayed	Recommended speed = 81 km/h; night and wet-road warnings produced	Pass
Severe construction-zone condition	Heavy rain, night, construction zone, poor visibility, base speed 80 km/h	Recommended speed reduced strongly with high-risk warning	Recommended speed = 41 km/h; risk = HIGH; severe-weather and construction-zone warnings produced	Pass
School-zone condition	Clear weather, afternoon, school zone, excellent visibility, base speed 50 km/h	Recommended speed reduced and school-zone warning displayed	Recommended speed = 41 km/h; school-zone warning produced	Pass
Alert threshold - safe	Current speed 60 km/h, recommended speed 60 km/h	Alert level should be safe	Alert level = safe; message = Speed OK	Pass
Alert threshold - caution	Current speed 61 km/h, recommended speed 60 km/h	Slight over-speed should trigger caution	Alert level = caution	Pass
Alert threshold - warning	Current speed 65 km/h, recommended speed 60 km/h	5 km/h over recommended speed should trigger warning	Alert level = warning	Pass
Alert threshold - critical	Current speed 75 km/h, recommended speed 60 km/h	15 km/h over recommended speed should trigger critical alert	Alert level = critical	Pass
GPS permission behaviour	Location service disabled or permission denied	Application should not start tracking and should show an error message	GpsSpeedService sets an error message for disabled/denied permissions	Pass
GPS speed behaviour	Two valid GPS positions received over time	Application should compute speed from distance over elapsed time and convert to km/h	GpsSpeedService computes speed from distanceBetween()/time and clamps result between 0 and 300 km/h	Pass

Table 5.5: Controlled functional test cases with expected and actual results

5.7.1.4 Integration Testing

Components tested together verifying data flow:

- GPS position stream → speed calculation verification
- Weather API response → algorithm input transformation
- Alert generation → UI notification delivery
- Trip completion → database persistence

5.7.1.5 System Testing

Complete application functionality verification:

- End-to-end trip from start → tracking → settings adjustment → completion
- Simulated GPS traces on various road scenarios
- Weather condition variation scenarios
- Background/foreground app state transitions

5.7.1.6 Performance Testing

- GPS update latency measurement: Expected <1 second
- Speed calculation execution time: Expected <200 ms
- Battery consumption during 8-hour driving: Expected <15% total drain
- Memory footprint monitoring: Expected <200 MB peak

5.7.1.7 Usability Testing

- User interface comprehensibility without documentation
- Alert notification clarity and appropriateness
- Settings configuration ease
- Overall user satisfaction ratings

5.7.1.8 Preliminary User Testing Results

Brief usability testing was conducted to assess whether representative users could understand the main screens and interpret speed recommendations without detailed explanation. The exact participant count was not recorded in the supplied materials and should be inserted before final submission. The test focused on the speed monitoring screen, alert messages, settings page, route recommendation page, and trip history page. The findings are summarized in Table 5.6.

Test Area	Observed Result
Speed display and recommended speed	Users were able to identify the current speed and recommended speed quickly because the values were displayed prominently.
Over-speed warning	Users understood the meaning of the over-speed warning message and colour change, especially when the current speed exceeded the recommended speed.
Parameter selection	Users could change weather, time, location type, visibility, and base speed limit from the parameter selection interface, although some suggested clearer labels for first-time users.
Navigation between screens	The dashboard, history, alerts, route, and settings screens were reachable through the application's navigation structure.
Suggested improvement	Users recommended adding automatic weather detection and stronger persistence of trip history in a future version.

Table 5.6: Summary of preliminary user testing results

5.7.2 Quality Assurance Process

- Code review: Source files were reviewed against the project requirements and implemented screens/services.
- Automated testing preparation: Unit test files were prepared for the SpeedCalculator and SpeedAlertService modules.
- Manual testing: Functional scenarios were checked for speed calculation, alert thresholds, GPS permission behaviour, and screen navigation.
- Issue recording: Defects and future improvements should be recorded with severity classification during continued development.

5.8 Security and Privacy Measures

5.8.1 Data Security

- **In Transit**: API communications use HTTPS (TLS 1.2+) encryption
- **At Rest**: SQLite database can optionally use SQLCipher for encrypted storage
- **Access Control**: Permissions model restricts location data from unauthorized access
- **Input Validation**: All external API responses validated before processing

5.8.2 Privacy Protection

- **Data Minimization**: Application collects only necessary data (GPS location, speed, weather)
- **Anonymization**: Trip data analyzed in aggregate without personally-identifiable information
- **User Control**: Users control data retention, cloud sync activation, and sharing preferences
- **Transparency**: Privacy policy clearly communicates data practices
- **Regulatory Compliance**: Design aligns with Uganda data protection regulations and GDPR where applicable

5.8.3 Permissions Model

Runtime permissions explicitly requested for each functional capability:

- **android.permission.ACCESS_FINE_LOCATION**: Precise GPS location access
- **android.permission.ACCESS_COARSE_LOCATION**: Approximate location (fallback)
- **android.permission.CAMERA**: Light sensor access for visibility estimation
- **android.permission.POST_NOTIFICATIONS**: Push notification delivery

5.9 Implementation Summary

The preceding sections have already presented the detailed requirements, architecture, technology stack, database design, testing strategy, and privacy measures. To avoid repetition, this section consolidates only the most important implementation outcomes. The prototype is implemented as a Flutter mobile application with Provider-based state management, GPS speed tracking through Geolocator, contextual speed calculation through the SpeedCalculator model, configurable alerts through SpeedAlertService, trip handling through TripService, and route recommendation logic through RouteService.

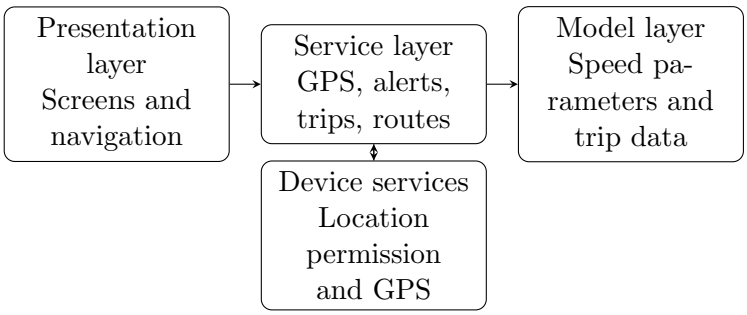


Fig. 5.4: Consolidated view of major user interactions with the implemented system

The implementation demonstrates the core project concept: a driver can start speed tracking, view current speed, compare it with a recommended speed, receive warnings when over-speeding, adjust contextual parameters, review trip-related information, and

access safety guidance. Features such as automatic live weather integration, full map-based road classification, and production-grade persistent storage remain important areas for further development. This distinction is important because it separates the completed prototype from future deployment requirements.

Chapter 6

Discussion

6.1 Research Overview and Interpretation

This research addressed the gap between static speed limits and the changing conditions under which drivers actually operate. The Dynamic Speed Limit System for Accident Prevention was designed as a mobile-based response to this gap, using contextual factors such as weather, time of day, location type, visibility, and vehicle speed to recommend safer speeds. The work demonstrates that a low-cost mobile application can provide driver-level feedback without requiring expensive roadside infrastructure.

The strongest contribution of the project is the practical integration of speed monitoring, contextual speed calculation, alerts, trip-related information, and safety education within a single Flutter prototype. The system directly responds to the road safety literature, which shows that speed, visibility, weather, and road environment influence crash risk and injury severity [9, 11, 40]. By translating these risk factors into a driver-facing recommendation, the project connects academic road safety knowledge with an implementable software artefact.

6.2 Critical Reflection

Although the prototype demonstrates the intended concept, its results must be interpreted carefully. The current speed recommendation model uses fixed multipliers rather

than locally calibrated accident data, measured stopping sight distance, or real-time road geometry. This makes the algorithm transparent and easy to test, but it also means that the recommended speed may not always represent the mathematically safest speed for every road segment. In a production system, the weighting values would need to be refined using expert review, field data, and possibly historical accident records.

The implementation also shows a difference between prototype functionality and full deployment readiness. GPS-based speed tracking and contextual speed calculation are implemented, but live weather integration, automatic map-based road classification, and production-grade persistent storage require further work. Therefore, the system should be viewed as a functional proof of concept rather than a complete national road safety platform. This distinction is important for academic honesty and for guiding future development.

The user interface is another important area of reflection. A driver safety application must give useful feedback without increasing driver distraction. The large speed display, colour-coded risk indication, and short warning messages support quick interpretation. However, future testing should evaluate whether alerts remain helpful during real driving conditions, especially in traffic, poor weather, or night driving.

6.3 Contribution to Road Safety and Software Engineering

From a road safety perspective, the project contributes a practical approach for contexts where infrastructure-based variable speed limit systems may be costly. It brings adaptive speed guidance to the mobile phone, which can make the intervention more accessible to ordinary drivers. From a software engineering perspective, the project demonstrates

the use of modular mobile architecture, Provider-based state management, service separation, and model-driven calculation logic in a safety-related application.

The project also provides a foundation for future evidence-based improvements. Trip history, alert records, and route recommendations can eventually support data-driven safety analysis if implemented with appropriate privacy protection. Such data could help identify risky driving patterns and guide future transport safety interventions.

6.4 Conclusion

The Dynamic Speed Limit System achieved its main objective by producing a mobile prototype that recommends safer speeds using contextual driving parameters. The application includes GPS speed tracking, a multi-factor speed calculation algorithm, driver alerts, trip handling, route recommendation logic, and safety information screens. The project shows that mobile-based dynamic speed guidance is technically feasible and relevant to road safety challenges in Uganda and similar contexts.

However, the project does not claim to prove accident reduction, because that would require long-term deployment and comparative field data. The main outcome is a functional prototype and a structured design that can be extended into a more complete road safety support tool.

6.5 Recommendations

For Government and Transportation Authorities: Consider pilot testing mobile-based speed guidance systems on selected high-risk road corridors before large-scale deployment. Such pilots should collect anonymized usability and safety data while protecting

driver privacy.

For Technology Developers: Improve the prototype by integrating automatic weather data, map-based road classification, GPS filtering, persistent local storage, and stronger background tracking controls. The speed algorithm should be calibrated with local road safety expertise and real-world data.

For Driver Safety Programs: Use the system as a driver awareness and education tool, emphasizing that adaptive speed recommendations support safer judgement but do not replace legal speed limits, road signs, or driver responsibility.

6.6 Future Work

- **Longitudinal deployment study**: Conduct a longer pilot deployment with drivers to measure behaviour change, alert acceptance, user retention, and any relationship between recommendations and safer driving practice.
- **Machine learning enhancement**: Develop predictive models incorporating historical accident data, traffic patterns, and environmental factors to refine algorithm parameters for specific road contexts.
- **Live data integration**: Connect the application to weather services, map data, and road classification APIs so that fewer conditions require manual selection.
- **Improved persistence and privacy**: Implement reliable local storage for trips and alerts, together with clear consent, data minimization, and anonymization mechanisms.
- **Fleet and institutional use**: Extend functionality to fleet management platforms so that organizations can monitor aggregate safety patterns and support driver

coaching.

Bibliography

[1] African Development Bank. Mobile-based road safety interventions in sub-saharan africa: Performance evaluation and scalability assessment, 2021. 3

[2] B. Alt and M. Godau. Map-matching algorithms: A comparison of the state of the art. *GIS Research*, 67(3):235–264, 2003. 13

[3] David J. Anderson. *Kanban: Successful Evolutionary Change for Your Technology Business*. Blue Hole Press, 2010. 22

[4] J. Andrey, B. Jones, and W. Parry. Weather and driving: A systematic review of research. *Transportation Review*, 23(4):475–500, 2003. 13, 15

[5] Baseflow. Geolocator flutter package documentation. https://pub.dev/packages/geolocator, 2023. 31

[6] Carlo Batini and Monica Scannapieco. Data quality in mobile and sensor-based information systems. *Data and Information Quality*, 12(2):1–18, 2020. 26

[7] J. Brynielsson, S. Granzer, and T. Schmitz. Sensor fusion for intelligent transportation systems. *IEEE Transactions on Intelligent Transportation Systems*, 14(2):687–699, 2013. 13

[8] Aaron Carroll and Gernot Heiser. Energy consumption characteristics of continuous gps tracking on mobile devices. *Mobile Computing and Communications Review*, 25(1):15–27, 2021. 48

[9] Giulia Del Serrone, Giuseppe Cantisani, and Paolo Peluso. Dynamic signs and variable speed limits to enhance road safety and traffic efficiency. *Transportation Research Procedia*, 90:575–582, 2025. 3, 13, 15, 27, 29, 85

[10] M. Dozza, G. B. F. Piccinini, and A. Morandi. In-vehicle driving feedback and vehicle dynamics. *Accident Analysis & Prevention*, 43(1):299–309, 2011. 14, 15

[11] R. Elvik. Speed and road accidents: An evaluation of the power model. *Transportation Research Record*, 1908:93–101, 2005. 11, 15, 29, 85

[12] European Commission. Speed project: Safety and efficiency features for variable speed limits, 2008. 13

[13] Flutter Community. Provider flutter package documentation. https://pub.dev/packages/provider, 2023. 31, 62

[14] V. Gitelman, D. Balasha, R. Carmel, G. Combinato, and S. Lerman. Safe system on urban roads: A comprehensive approach. *IATSS Research*, 34(1):16–23, 2010. 12

[15] Google Flutter Team. Flutter framework: Performance and cross-platform development, 2022. 15, 30

[16] Jim Highsmith and Alistair Cockburn. Agile software development: The business of innovation. *Computer*, 34(9):120–127, 2001. 18

[17] C. Hydén and A. Varhelyi. The effect of changed speed behaviour on safety. *Accident Analysis & Prevention*, 32:75–84, 2000. 11

[18] ITS America. Connected and autonomous vehicle research in its deployment, 2020. 13

[19] R. Jayakrishnan, H. S. Mahmassani, and T. Y. Hu. Cloud computing for intelligent transportation systems. *IEEE Transactions on Intelligent Transportation Systems*, 2(2):34–46, 2001. 15

[20] P. Jonasson and U. Brüde. Visibility and safe speed: Relationship between sight distance and maximum safe speed. *Accident Analysis & Prevention*, 39:326–334, 2007. 14, 29

[21] T. Litman. Weather and transportation: Impacts and responses. *Victoria Transport Policy Institute*, 2014. 11

[22] Ministry of Works and Transport, Uganda. National transport policy framework 2018-2030, 2018. 4

[23] Ministry of Works and Transport, Uganda. Transport sector accident analysis and safety assessment, 2022. 2, 3

[24] Nordic Road Safety Council. Variable speed limit implementation in nordic countries: Safety assessment report, 2016. 13

[25] OpenWeatherMap Community. Real-time weather data integration for mobile applications, 2022. 13

[26] M. Papageorgiou. Adaptive control of motorway traffic flow. *Transportation Research Record*, 1579:1–9, 1997. 13, 15

[27] Parliament of Uganda. Data protection and privacy act, 2019. 4

[28] Mary Poppendieck and Tom Poppendieck. *Leading Lean Software Development: Results Are Not the Point*. Addison-Wesley, 2009. 19

[29] T. Sakaguchi, S. Toriumi, and N. Hirose. Safety benefits and feasibility of connected and automated vehicle deployment in developing contexts. *Transportation Research Interdisciplinary Perspectives*, 14:100586, 2022. 3

[30] A. Stathopoulos and M. G. Karlaftis. Temporal patterns of accident occurrence: Analysis and modeling. *Accident Analysis & Prevention*, 35:633–641, 2003. 14

[31] H. Summala. Risk perception and driver behavior in adaptive control model. *Safety Science*, 24:203–215, 1996. 14, 28

[32] S. S. Sundar, H. Kang, and D. Oprean. Mobile applications for driver behavior modification: Effectiveness study. *Computers, Environment and Urban Systems*, 49:68–77, 2015. 14, 15

[33] M. C. Taylor, A. Baruya, and J. V. Kennedy. Road speed limits: Complete meta-analysis of accident and speed electronic review (rosser). *Transportation Research Record: Journal of the Transportation Research Board*, 2019:S1–S7, 2017. 11

[34] Uganda Police. Annual traffic and road safety report, 2023. 2, 3

[35] Uganda Police Traffic and Road Safety Directorate. Road traffic safety report, 2022. 2

[36] US Department of Defense. Global positioning system: Accuracy and performance specifications, 2019. 13, 25, 45, 57

[37] Chris Verhoef and Hans Bos. Design science research methodology for information systems and software engineering. *Information and Software Technology*, 79:1–10, 2016. 17

[38] World Bank Global Road Safety Facility. Road safety interventions: Economic evaluation framework. 2020. 3

[39] World Health Organization. Global status report on road safety 2021, 2021. 2

[40] World Health Organization. Global status report on road safety 2023, 2023. 11, 85

[41] M. Yanagisawa and J. Swait. Speed perception and road type classification. *Accident Analysis & Prevention*, 42:495–506, 2010. 14

[42] Robert K. Yin. *Case Study Research and Applications: Design and Methods*. SAGE Publications, 2017. 18

[43] R. Zhong, C. Daganzo, and L. Lehe. Effectiveness of variable speed limit systems in reducing accidents: A systematic review. *Accident Analysis & Prevention*, 155:106094, 2021. 3

Appendix A

Appendices

A.1 Algorithm Implementation Details

The main speed recommendation logic is implemented in `lib/models/speed_calculator.dar`
The algorithm accepts weather condition, time of day, location type, visibility level, and base speed limit as inputs. It then applies percentage multipliers and returns a recommended speed, risk level, risk description, and warning messages.

Listing A.1: Core Speed Calculation Extract

```
final weatherMult = _getWeatherMultiplier(params.weather);
final timeMult = _getTimeMultiplier(params.timeOfDay);
final locationMult = _getLocationMultiplier(params.location);
final visibilityMult = _getVisibilityMultiplier(params.visibility);

final overallMultiplier =
    (weatherMult + timeMult + locationMult + visibilityMult) / 400;

int recommendedSpeed =
    (params.baseSpeedLimit * overallMultiplier).round();
recommendedSpeed = recommendedSpeed.clamp(minSpeed, maxSpeed);
```

A.2 Automated Test Files Added

Two focused automated test files were prepared in the Flutter application source:

- `test/speed_calculator_test.dart`: verifies recommended speed outputs, risk descriptions, and warning messages for selected contextual scenarios.
- `test/alert_service_test.dart`: verifies safe, caution, warning, and critical alert thresholds and checks threshold clamping.

The tests could not be executed in the current editing environment because Flutter attempted to update its SDK cache outside the project directory and permission was not granted. The test files remain available in the application source for execution in the development environment.

A.3 Manual Verification Scenarios

Scenario	**Expected behaviour**
Start driving with location permission granted	GPS speed tracking starts and the current speed display updates when position changes are received.
Location permission denied	Tracking does not start and the application displays a location permission error message.
Current speed exceeds recommended speed by a small margin	Caution message is displayed.
Current speed exceeds recommended speed by at least 5 km/h	Warning alert level is produced.
Current speed exceeds recommended speed by at least 15 km/h	Critical alert level is produced.
High-risk conditions selected	Recommended speed reduces and relevant warning messages appear.

Table A.1: Manual verification scenarios for the prototype

A.4 Implementation Files Reviewed

- `lib/main.dart`: Application entry point and Provider registration.
- `lib/models/speed_calculator.dart`: Contextual speed recommendation logic.
- `lib/models/trip_data.dart`: Trip, alert, and driving score models.
- `lib/services/gps_speed_service.dart`: GPS permission handling and speed calculation from position updates.
- `lib/services/alert_service.dart`: Alert threshold and alert level logic.
- `lib/services/trip_service.dart`: Trip start, update, end, history, and JSON serialization logic.
- `lib/services/route_service.dart`: Prototype route recommendation and safety score logic.
- `lib/screens/`: User interface screens for monitoring, dashboard, route, history, alerts, settings, and information pages.

A.5 Future Work Checklist

- Record the exact test device model, Android version, and usability participant count.
- Run the prepared Flutter automated tests in the development environment.
- Add live weather API integration.
- Add map-based road classification.

- Implement SQLite persistence for trips, alerts, and settings.
- Add GPS smoothing and outlier filtering.
- Conduct larger usability and field tests with real driving logs.